

STAR RX72N - rx_frame_ascii Library
1.0.0

Generated by Doxygen 1.16.1

1 Topic Index	1
1.1 Topics	1
2 Directory Hierarchy	3
2.1 Directories	3
3 File Index	5
3.1 File List	5
4 Topic Documentation	7
4.1 Internal Helper Functions	7
4.1.1 Detailed Description	8
4.1.2 Function Documentation	8
4.1.2.1 internal_append_char()	8
4.1.2.2 internal_append_decimal_u16()	10
4.1.2.3 internal_append_flag()	11
4.1.2.4 internal_append_hex_byte()	14
4.1.2.5 internal_append_hex_u16()	15
4.1.2.6 internal_append_hex_u32()	17
4.1.2.7 internal_append_str()	18
4.1.2.8 internal_format_header()	20
4.1.2.9 internal_format_payload()	22
4.1.2.10 internal_is_printable()	24
5 Directory Documentation	27
5.1 libs/rx_frame_ascii/inc Directory Reference	27
5.2 libs Directory Reference	27
5.3 libs/rx_frame_ascii Directory Reference	28
5.4 libs/rx_frame_ascii/src Directory Reference	28
6 File Documentation	29
6.1 libs/rx_frame_ascii/inc/rx_frame_ascii.h File Reference	29
6.1.1 Detailed Description	30
6.1.2 Enumeration Type Documentation	33
6.1.2.1 rx_frame_ascii_constants_t	33
6.1.3 Function Documentation	35
6.1.3.1 rx_frame_ascii_flags_str()	35
6.1.3.2 rx_frame_ascii_format()	38
6.1.3.3 rx_frame_ascii_type_name()	43
6.2 rx_frame_ascii.h	46
6.3 libs/rx_frame_ascii/src/rx_frame_ascii.c File Reference	52
6.3.1 Detailed Description	54
6.3.2 Enumeration Type Documentation	56
6.3.2.1 buffer_size_constants_t	56

6.3.2.2 decimal_buf_constants_t	57
6.3.2.3 format_constants_t	58
6.3.3 Function Documentation	60
6.3.3.1 rx_frame_ascii_flags_str()	60
6.3.3.2 rx_frame_ascii_format()	63
6.3.3.3 rx_frame_ascii_type_name()	64
6.3.4 Variable Documentation	65
6.3.4.1 s_hex_chars	65
6.4 rx_frame_ascii.c	66

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Internal Helper Functions	7
-------------------------------------	---

Chapter 2

Directory Hierarchy

2.1 Directories

inc	27
rx_frame_ascii.h	29
libs	27
rx_frame_ascii	28
inc	27
rx_frame_ascii.h	29
src	28
rx_frame_ascii.c	52
rx_frame_ascii	28
inc	27
rx_frame_ascii.h	29
src	28
rx_frame_ascii.c	52
src	28
rx_frame_ascii.c	52

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

libs/rx_frame_ascii/inc/ rx_frame_ascii.h	
Frame ASCII Formatter for Human-Readable Frame Dumps	29
libs/rx_frame_ascii/src/ rx_frame_ascii.c	
Frame ASCII Formatter Implementation	52

Chapter 4

Topic Documentation

4.1 Internal Helper Functions

String building helper functions with bounds checking.

Functions

- static uint32_t [internal_append_str](#) (char *buf, uint32_t pos, uint32_t max, const char *str)
Append string to buffer with bounds checking.
- static uint32_t [internal_append_char](#) (char *buf, uint32_t pos, uint32_t max, char c)
Append single character to buffer with bounds checking.
- static uint32_t [internal_append_hex_byte](#) (char *buf, uint32_t pos, uint32_t max, uint8_t byte)
Append byte as two hex characters to buffer.
- static uint32_t [internal_append_hex_u16](#) (char *buf, uint32_t pos, uint32_t max, uint16_t value)
Append uint16_t as four hex characters to buffer.
- static uint32_t [internal_append_hex_u32](#) (char *buf, uint32_t pos, uint32_t max, uint32_t value)
Append uint32_t as eight hex characters to buffer.
- static uint32_t [internal_append_decimal_u16](#) (char *buf, uint32_t pos, uint32_t max, uint16_t value)
Append uint16_t as decimal digits to buffer.
- static bool [internal_is_printable](#) (uint8_t c)
Check if byte value is printable ASCII character.
- static uint32_t [internal_format_payload](#) (char *buf, uint32_t pos, uint32_t max, const uint8_t *payload, uint16_t length)
Format payload as hex dump with ASCII sidebar.
- static uint32_t [internal_format_header](#) (char *buf, uint32_t pos, uint32_t max, const rx_frame_header_t *header, bool is_tx)
Format frame header metadata as single text line.
- static uint32_t [internal_append_flag](#) (char *buffer, uint32_t pos, uint32_t max, bool *first, uint8_t flags, uint8_t flag_bit, const char *flag_name)
Append a single flag name with pipe separator if needed.

4.1.1 Detailed Description

String building helper functions with bounds checking.

All append helpers follow a consistent signature pattern for composability:

```
uint32_t internal_append_XXX(char* buf, uint32_t pos, uint32_t max, <value>);
```

Parameters

- **buf**: Output buffer pointer (may be NULL for safety)
- **pos**: Current write position in buffer
- **max**: Maximum buffer size (total capacity)
- **value**: Data to append (varies by function)

Return Value

All helpers return the updated position after appending. If the append would overflow, the original position is returned unchanged.

Bounds Safety

- All functions check $pos < max - k_null_terminator$ before writing
- NULL buffer pointer causes immediate return with unchanged position
- Zero-size buffer causes immediate return with unchanged position
- No writes beyond $max - 1$ to preserve space for null terminator

Usage Pattern

```
uint32_t pos = 0;
pos = internal_append_str(buf, pos, max, "Hello ");
pos = internal_append_decimal_u16(buf, pos, max, 42);
pos = internal_append_char(buf, pos, max, '!');
buf[pos] = '\0'; // Null terminate
// Result: "Hello 42!"
```

4.1.2 Function Documentation

4.1.2.1 internal_append_char()

```
uint32_t internal_append_char (
    char * buf,
    uint32_t pos,
    uint32_t max,
    char c) [static]
```

Append single character to buffer with bounds checking.

Writes a single character to the buffer at the current position if space is available (accounting for null terminator).

Parameters

in,out	<i>buf</i>	Output buffer (may be NULL)
--------	------------	-----------------------------

in	<i>pos</i>	Current write position
in	<i>max</i>	Maximum buffer size
in	<i>c</i>	Character to append

Returns

Updated position after appending

Return values

<i>pos</i>	(unchanged) If buf is nullptr or buffer full
<i>pos</i>	+ 1 If character was written

Precondition

$pos < max - 1$ (for writing to occur)

Postcondition

buf[pos] contains c (if written)

Since

Version 1.0.0

Definition at line 342 of file rx_frame_ascii.c.

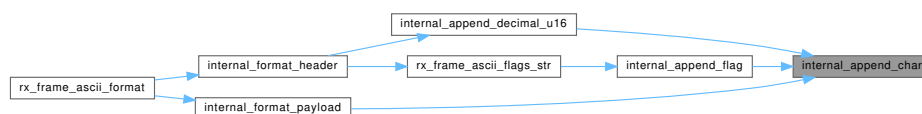
```

00343 {
00344     /* Rule 5: Pre-condition validation */
00345     if (buf == nullptr || max == k_empty_buffer || pos >= max) {
00346         return pos;
00347     }
00348     if (pos < max - k_null_terminator) {
00349         buf[pos++] = c;
00350     }
00351     return pos;
00352 }
00353 }
```

References [k_empty_buffer](#), and [k_null_terminator](#).

Referenced by [internal_append_decimal_u16\(\)](#), [internal_append_flag\(\)](#), and [internal_format_payload\(\)](#).

Here is the caller graph for this function:



4.1.2.2 internal_append_decimal_u16()

```
uint32_t internal_append_decimal_u16 (
    char * buf,
    uint32_t pos,
    uint32_t max,
    uint16_t value) [static]
```

Append uint16_t as decimal digits to buffer.

Converts a 16-bit value to its decimal string representation (1-5 digits). Uses a small stack buffer for digit reversal, avoiding heap allocation. Zero value outputs single '0' character.

Algorithm

1. Extract digits via repeated modulo/division (reverse order)
2. Store in temporary stack buffer
3. Reverse and append to output buffer

Parameters

in,out	<i>buf</i>	Output buffer (may be NULL)
in	<i>pos</i>	Current write position
in	<i>max</i>	Maximum buffer size
in	<i>value</i>	16-bit value to convert (0-65535)

Returns

Updated position after appending

Return values

<i>pos</i>	(unchanged) If buf is nullptr or insufficient space
------------	---

Precondition

Sufficient space for up to 5 digits

Postcondition

Decimal digits written without leading zeros (except for value=0)

Example Output

- value=0 -> "0"
- value=1 -> "1"
- value=12345 -> "12345"
- value=65535 -> "65535"

Stack Usage

~6 bytes (temp buffer for digit reversal)

Since

Version 1.0.0

Definition at line 514 of file [rx_frame_ascii.c](#).

```

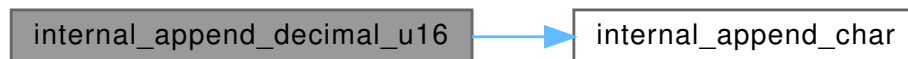
00515 {
00516     /* Rule 5: Pre-condition validation */
00517     if (buf == nullptr || max == k_empty_buffer || pos >= max) {
00518         return pos;
00519     }
00520
00521     char temp[k_u16_decimal_buf_size]; /* Max 5 digits + null for uint16_t */
00522     uint8_t len = 0;
00523
00524     if (value == 0) {
00525         return internal_append_char(buf, pos, max, '0');
00526     }
00527
00528     while (value > 0 && len < k_u16_decimal_max_digits) {
00529         const uint8_t digit = (uint8_t)(value % k_decimal_base);
00530         temp[len++] = (char)('0' + digit);
00531         value /= k_decimal_base;
00532     }
00533
00534     /* Reverse digits into output */
00535     while (len > 0) {
00536         pos = internal_append_char(buf, pos, max, temp[--len]);
00537     }
00538
00539     return pos;
00540 }

```

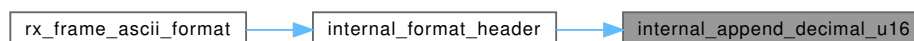
References [internal_append_char\(\)](#), [k_decimal_base](#), [k_empty_buffer](#), [k_u16_decimal_buf_size](#), and [k_u16_decimal_max_digits](#).

Referenced by [internal_format_header\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.3 internal_append_flag()

```

uint32_t internal_append_flag (
    char * buffer,
    uint32_t pos,
    uint32_t max,
    bool * first,
    uint8_t flags,
    uint8_t flag_bit,
    const char * flag_name) [static]

```

Append a single flag name with pipe separator if needed.

Conditionally appends a flag name to the output buffer if the corresponding flag bit is set. Manages separator insertion: no separator before first flag, pipe separator before subsequent flags.

Example Sequence

1. flags=ACK|PRI, first call (first=true): appends "ACK", sets first=false
2. Same flags, second call: appends "|PRI"

Parameters

in,out	<i>buffer</i>	Output buffer
--------	---------------	---------------

in	<i>pos</i>	Current position in buffer
in	<i>max</i>	Maximum buffer size
in,out	<i>first</i>	Pointer to first-flag indicator • true: No separator needed (this is first flag) • false: Prepend ' ' separator • Set to false after appending a flag
in	<i>flags</i>	Complete flag bitmask to test
in	<i>flag_bit</i>	Specific flag bit to check (e.g., 0x01 for ACK)
in	<i>flag_name</i>	String name of the flag (e.g., "ACK")

Returns

Updated position in buffer

Return values

<i>pos</i>	(unchanged) If <i>flag_bit</i> not set or <i>first</i> is nullptr
------------	---

Precondition

first != nullptr

Postcondition

**first* set to false if flag was appended

Separator '|' prepended if not first flag

Since

Version 1.0.0

Definition at line 834 of file [rx_frame_ascii.c](#).

```

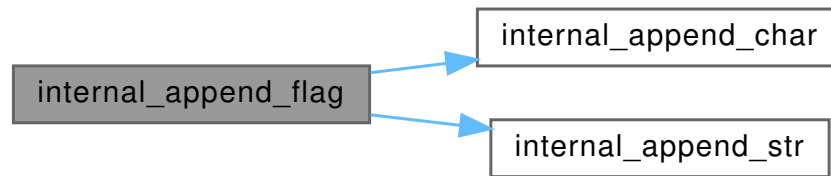
00841 {
00842     /* Rule 5: Pre-condition validation */
00843     if (first == nullptr) {
00844         return pos;
00845     }
00846
00847     if (flags & flag_bit) {
00848         if (!*first) {
00849             pos = internal_append_char(buffer, pos, max, '|');
00850         }
00851         pos = internal_append_str(buffer, pos, max, flag_name);
00852         *first = false;
00853     }
00854     return pos;
00855 }

```

References [internal_append_char\(\)](#), and [internal_append_str\(\)](#).

Referenced by [rx_frame_ascii_flags_str\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.4 internal_append_hex_byte()

```

uint32_t internal_append_hex_byte (
    char * buf,
    uint32_t pos,
    uint32_t max,
    uint8_t byte) [static]
  
```

Append byte as two hex characters to buffer.

Converts a single byte to its two-character hexadecimal representation (e.g., 0xFF -> "FF") and appends it to the buffer. Uses uppercase hex letters via s_hex_chars lookup table.

Parameters

in,out	<i>buf</i>	Output buffer (may be NULL)
in	<i>pos</i>	Current write position
in	<i>max</i>	Maximum buffer size
in	<i>byte</i>	Byte value to convert (0x00-0xFF)

Returns

Updated position after appending (pos + 2 on success)

Return values

<i>pos</i>	(unchanged) If buf is nullptr or insufficient space
------------	---

Precondition

pos + 2 <= max - 1 (for writing to occur)

Postcondition

buf[pos] contains upper nibble hex char
 buf[pos+1] contains lower nibble hex char

Example Output

- byte=0x00 -> "00"
- byte=0xFF -> "FF"
- byte=0x5A -> "5A"

Since

Version 1.0.0

Definition at line 384 of file `rx_frame_ascii.c`.

```
00385 {
00386  /* Rule 5: Pre-condition validation */
00387  if (buf == nullptr || max < k_hex_chars_per_byte || pos >= max) {
00388      return pos;
00389  }
00390
00391  if (pos < max - k_hex_chars_per_byte) {
00392      buf[pos++] = s_hex_chars[(byte » k_hex_digit_shift) & k_hex_digit_mask];
00393      buf[pos++] = s_hex_chars[byte & k_hex_digit_mask];
00394  }
00395  return pos;
00396 }
```

References `k_hex_chars_per_byte`, `k_hex_digit_mask`, `k_hex_digit_shift`, and `s_hex_chars`.

Referenced by `internal_append_hex_u16()`, and `internal_format_payload()`.

Here is the caller graph for this function:



4.1.2.5 internal_append_hex_u16()

```
uint32_t internal_append_hex_u16 (
    char * buf,
    uint32_t pos,
    uint32_t max,
    uint16_t value) [static]
```

Append `uint16_t` as four hex characters to buffer.

Converts a 16-bit value to its four-character hexadecimal representation (e.g., 0xABCD -> "ABCD") in big-endian order (most significant byte first).

Parameters

in,out	<i>buf</i>	Output buffer (may be NULL)
in	<i>pos</i>	Current write position
in	<i>max</i>	Maximum buffer size
in	<i>value</i>	16-bit value to convert

Returns

Updated position after appending (pos + 4 on success)

Return values

<i>pos</i>	(unchanged) If buf is nullptr or insufficient space
------------	---

Precondition

pos + 4 <= max - 1 (for writing to occur)

Postcondition

Four hex characters written at buf[pos..pos+3]

Example Output

- value=0x0000 -> "0000"
- value=0xFFFF -> "FFFF"
- value=0x1234 -> "1234"

Since

Version 1.0.0

Definition at line 425 of file rx_frame_ascii.c.

```

00426 {
00427     /* Rule 5: Pre-condition validation */
00428     if (buf == nullptr || max < k_hex_chars_per_u16 || pos >= max) {
00429         return pos;
00430     }
00431
00432     pos = internal_append_hex_byte(buf, pos, max, (uint8_t)(value » 8));
00433     pos = internal_append_hex_byte(buf, pos, max, (uint8_t)(value & k_mask_u8));
00434     return pos;
00435 }

```

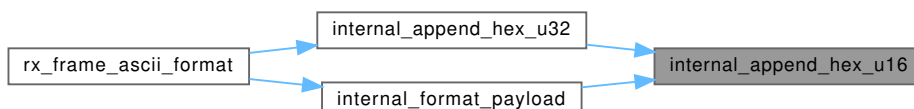
References [internal_append_hex_byte\(\)](#), and [k_mask_u8](#).

Referenced by [internal_append_hex_u32\(\)](#), and [internal_format_payload\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.6 internal_append_hex_u32()

```
uint32_t internal_append_hex_u32 (  
    char * buf,  
    uint32_t pos,  
    uint32_t max,  
    uint32_t value) [static]
```

Append uint32_t as eight hex characters to buffer.

Converts a 32-bit value to its eight-character hexadecimal representation (e.g., 0xDEADBEEF -> "DEADBEEF") in big-endian order. Commonly used for CRC-32 values in frame output.

Parameters

in,out	<i>buf</i>	Output buffer (may be NULL)
in	<i>pos</i>	Current write position
in	<i>max</i>	Maximum buffer size
in	<i>value</i>	32-bit value to convert

Returns

Updated position after appending (pos + 8 on success)

Return values

<i>pos</i>	(unchanged) If buf is nullptr or insufficient space
------------	---

Precondition

$pos + 8 \leq max - 1$ (for writing to occur)

Postcondition

Eight hex characters written at buf[pos..pos+7]

Example Output

- value=0x00000000 -> "00000000"
- value=0xFFFFFFFF -> "FFFFFFFF"
- value=0xDEADBEEF -> "DEADBEEF"

Since

Version 1.0.0

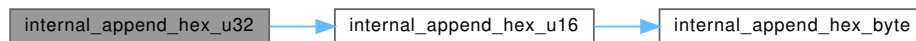
Definition at line 465 of file [rx_frame_ascii.c](#).

```
00466 {
00467  /* Rule 5: Pre-condition validation */
00468  if (buf == nullptr || max < k_hex_chars_per_u32 || pos >= max) {
00469    return pos;
00470  }
00471
00472  pos = internal_append_hex_u16(buf, pos, max, (uint16_t)(value » k_u32_high_u16_shift));
00473  pos = internal_append_hex_u16(buf, pos, max, (uint16_t)(value & k_mask_u16));
00474  return pos;
00475 }
```

References [internal_append_hex_u16\(\)](#), [k_mask_u16](#), and [k_u32_high_u16_shift](#).

Referenced by [rx_frame_ascii_format\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.7 internal_append_str()

```
uint32_t internal_append_str (
    char * buf,
    uint32_t pos,
    uint32_t max,
    const char * str) [static]
```

Append string to buffer with bounds checking.

Copies characters from source string to output buffer until null terminator is reached or buffer is full (leaving room for final null terminator).

Parameters

in,out	<i>buf</i>	Output buffer (may be NULL)
in	<i>pos</i>	Current write position
in	<i>max</i>	Maximum buffer size
in	<i>str</i>	Source string to append (may be NULL)

Returns

Updated position after appending

Return values

<i>pos</i>	(unchanged) If buf, str is nullptr, or buffer full
------------	--

Precondition

$pos < max$ (for any writing to occur)

Postcondition

Characters written in range [pos, return_value)

Buffer NOT null-terminated (caller responsibility)

Note

Does not write null terminator

Returns original pos if nothing can be written

Since

Version 1.0.0

Definition at line 306 of file [rx_frame_ascii.c](#).

```

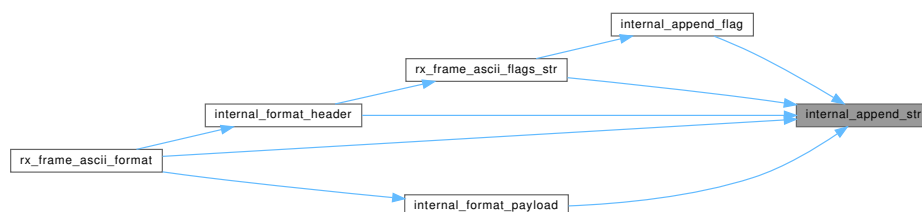
00307 {
00308     /* Rule 5: Pre-condition validation */
00309     if (buf == nullptr || str == nullptr || max == k_empty_buffer || pos >= max) {
00310         return pos;
00311     }
00312     while (*str != '\0' && pos < max - k_null_terminator) {
00313         buf[pos++] = *str++;
00314     }
00315     return pos;
00316 }
00317

```

References [k_empty_buffer](#), and [k_null_terminator](#).

Referenced by [internal_append_flag\(\)](#), [internal_format_header\(\)](#), [internal_format_payload\(\)](#), [rx_frame_ascii_flags_str\(\)](#), and [rx_frame_ascii_format\(\)](#).

Here is the caller graph for this function:



4.1.2.8 internal_format_header()

```
uint32_t internal_format_header (
    char * buf,
    uint32_t pos,
    uint32_t max,
    const rx_frame_header_t * header,
    bool is_tx)  [static]
```

Format frame header metadata as single text line.

Formats the frame header fields into a human-readable summary line including direction indicator, sequence number, payload length, frame type, and flags.

Output Format

```
[TX] seq=12345 len=64 type=COMMAND flags=ACK|PRI
[RX] seq=12346 len=128 type=RESPONSE flags=NONE
```

Field Details

Field	Format	Example
Direction	[TX] or [RX]	[TX]
Sequence	seq=N (decimal)	seq=12345
Length	len=N (decimal)	len=64
Type	type=NAME	type=COMMAND
Flags	flags=F1 F2	flags=ACK PRI

Parameters

in,out	<i>buf</i>	Output buffer
in	<i>pos</i>	Current position in buffer
in	<i>max</i>	Maximum buffer size
in	<i>header</i>	Frame header to format
in	<i>is_tx</i>	Direction indicator (true=TX, false=RX)

Returns

Updated position in buffer

Return values

<i>pos</i>	(unchanged) If buf or header is nullptr, or max is 0
------------	--

Precondition

buf != nullptr
header != nullptr

Postcondition

Header line written with CRLF line ending
pos <= max (invariant maintained)

Since

Version 1.0.0

Definition at line 713 of file [rx_frame_ascii.c](#).

```

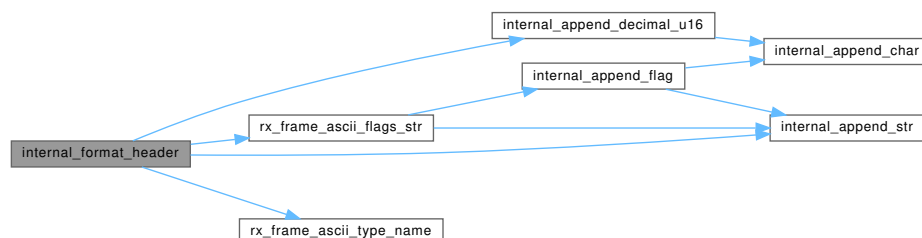
00718 {
00719     /* Rule 5: Pre-condition validation */
00720     if (buf == nullptr || header == nullptr || max == k_empty_buffer || pos >= max) {
00721         return pos;
00722     }
00723
00724     char flags_buf[k_flags_buf_size];
00725
00726     /* Direction */
00727     pos = internal_append_str(buf, pos, max, is_tx ? "[TX] " : "[RX] ");
00728
00729     /* Sequence number */
00730     pos = internal_append_str(buf, pos, max, "seq=");
00731     pos = internal_append_decimal_u16(buf, pos, max, header->sequence);
00732
00733     /* Length */
00734     pos = internal_append_str(buf, pos, max, " len=");
00735     pos = internal_append_decimal_u16(buf, pos, max, header->length);
00736
00737     /* Type */
00738     pos = internal_append_str(buf, pos, max, " type=");
00739     pos = internal_append_str(buf, pos, max, rx_frame_ascii_type_name((rx_frame_type_t)header->type));
00740
00741     /* Flags */
00742     pos = internal_append_str(buf, pos, max, " flags=");
00743     (void)rx_frame_ascii_flags_str(header->flags, flags_buf, sizeof(flags_buf));
00744     pos = internal_append_str(buf, pos, max, flags_buf);
00745     pos = internal_append_str(buf, pos, max, "\r\n");
00746
00747     /* Rule 5: Post-condition validation - ensure pos never exceeds max.
00748     * Defensive invariant: internal_append_* currently enforce pos <= max, but this
00749     * guard protects against buffer overflows if future refactors change their behavior. */
00750     if (pos > max) {
00751         pos = max;
00752     }
00753
00754     return pos;
00755 }

```

References [internal_append_decimal_u16\(\)](#), [internal_append_str\(\)](#), [k_empty_buffer](#), [k_flags_buf_size](#), [rx_frame_ascii_flags_str\(\)](#), and [rx_frame_ascii_type_name\(\)](#).

Referenced by [rx_frame_ascii_format\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.9 internal_format_payload()

```
uint32_t internal_format_payload (
    char * buf,
    uint32_t pos,
    uint32_t max,
    const uint8_t * payload,
    uint16_t length) [static]
```

Format payload as hex dump with ASCII sidebar.

Formats binary payload data as a traditional hex dump with 16 bytes per line, offset prefix, and ASCII sidebar showing printable characters (or '.' for non-printable bytes).

Output Format

Each line follows this structure:

```
[NNNN] XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX .....

```

Where:

- [NNNN] is the 4-digit hex offset from payload start
- XX XX ... is the hex representation of up to 16 bytes
- ASCII sidebar shows printable chars or '.' for non-printable

Empty Payload Handling

If length is 0, outputs: [PAYLOAD] (empty)\r\n

Parameters

in,out	<i>buf</i>	Output buffer
in	<i>pos</i>	Current position in buffer
in	<i>max</i>	Maximum buffer size
in	<i>payload</i>	Payload data to format (may be NULL if length is 0)
in	<i>length</i>	Payload length in bytes

Returns

Updated position in buffer

Return values

<i>pos</i>	(unchanged) If buf is nullptr or max is 0
------------	---

Precondition

buf != nullptr (for any output)

payload != nullptr if length > 0

Postcondition

Hex dump lines written with CRLF line endings

Loop Bounds (NASA Rule 2)

- Outer loop: bounded by `length / k_bytes_per_line + 1` iterations
- Inner loops: bounded by `k_bytes_per_line` (16) iterations

Since

Version 1.0.0

Definition at line 614 of file `rx_frame_ascii.c`.

```

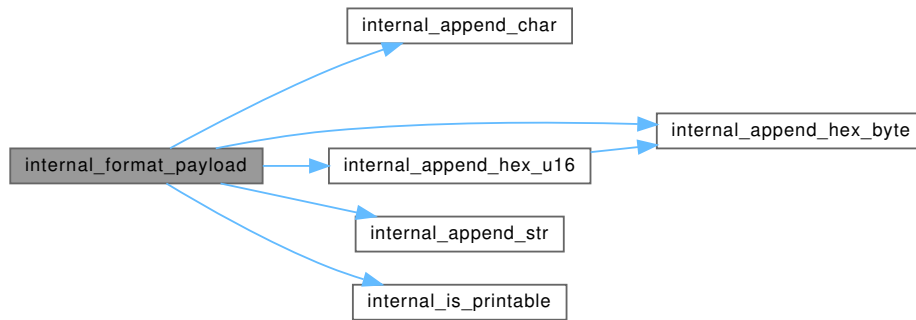
00619 {
00620     /* Rule 5: Pre-condition validation */
00621     if (buf == nullptr || max == k_empty_buffer || pos >= max) {
00622         return pos;
00623     }
00624
00625     if (length == 0) {
00626         pos = internal_append_str(buf, pos, max, "[PAYLOAD] (empty)\r\n");
00627         return pos;
00628     }
00629
00630     /* Validate payload is non-NULL if length > 0 */
00631     if (payload == nullptr) {
00632         return pos;
00633     }
00634
00635     uint16_t offset = 0;
00636     while (offset < length) {
00637         /* Start new line with offset */
00638         pos = internal_append_str(buf, pos, max, "[");
00639         pos = internal_append_hex_u16(buf, pos, max, offset);
00640         pos = internal_append_str(buf, pos, max, "] ");
00641
00642         /* Calculate bytes on this line */
00643         uint16_t line_bytes = length - offset;
00644         if (line_bytes > k_bytes_per_line) {
00645             line_bytes = k_bytes_per_line;
00646         }
00647
00648         /* Hex dump */
00649         for (uint16_t i = 0; i < k_bytes_per_line; i++) {
00650             if (i < line_bytes) {
00651                 pos = internal_append_hex_byte(buf, pos, max, payload[offset + i]);
00652                 pos = internal_append_char(buf, pos, max, ' ');
00653             } else {
00654                 pos = internal_append_str(buf, pos, max, " "); /* Pad short lines */
00655             }
00656         }
00657
00658         /* ASCII sidebar */
00659         pos = internal_append_char(buf, pos, max, '|');
00660         for (uint16_t i = 0; i < line_bytes; i++) {
00661             uint8_t c = payload[offset + i];
00662             pos = internal_append_char(buf, pos, max, internal_is_printable(c) ? (char)c : '.');
00663         }
00664
00665         pos = internal_append_str(buf, pos, max, "\r\n");
00666         offset += line_bytes;
00667     }
00668
00669     return pos;
00670 }

```

References `internal_append_char()`, `internal_append_hex_byte()`, `internal_append_hex_u16()`, `internal_append_str()`, `internal_is_printable()`, `k_bytes_per_line`, and `k_empty_buffer`.

Referenced by `rx_frame_ascii_format()`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.10 internal_is_printable()

```
bool internal_is_printable (
    uint8_t c) [static]
```

Check if byte value is printable ASCII character.

Tests whether a byte value falls within the printable ASCII range (0x20-0x7E inclusive). Used to decide whether to display the actual character or a '.' placeholder in hex dump ASCII sidebars.

Printable Range

- 0x20 (space) through 0x7E (~) are printable
- 0x00-0x1F are control characters (not printable)
- 0x7F (DEL) is not printable
- 0x80-0xFF are extended ASCII (not printable in this implementation)

Parameters

in	c	Byte value to test
----	---	--------------------

Returns

true if printable, false otherwise

Return values

<i>true</i>	If c in range [0x20, 0x7E]
<i>false</i>	Otherwise

Note

Pure function with no side effects

Since

Version 1.0.0

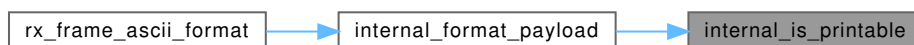
Definition at line 567 of file [rx_frame_ascii.c](#).

```
00568 {  
00569     return (c >= k_printable_min && c <= k_printable_max);  
00570 }
```

References [k_printable_max](#), and [k_printable_min](#).

Referenced by [internal_format_payload\(\)](#).

Here is the caller graph for this function:

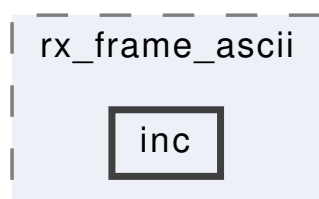


Chapter 5

Directory Documentation

5.1 libs/rx_frame_ascii/inc Directory Reference

Directory dependency graph for inc:

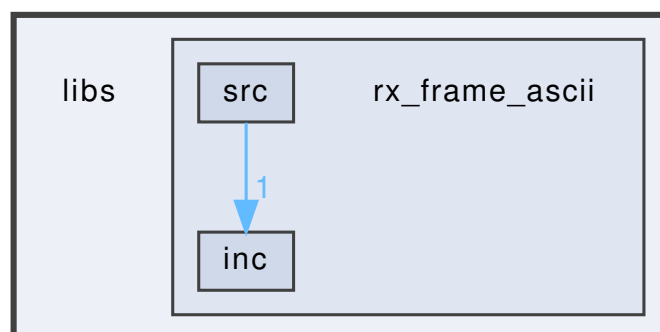


Files

- file [rx_frame_ascii.h](#)
Frame ASCII Formatter for Human-Readable Frame Dumps.

5.2 libs Directory Reference

Directory dependency graph for libs:

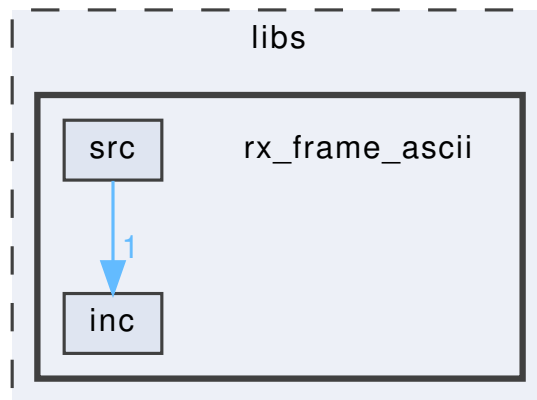


Directories

- directory [rx_frame_ascii](#)

5.3 libs/rx_frame_ascii Directory Reference

Directory dependency graph for rx_frame_ascii:

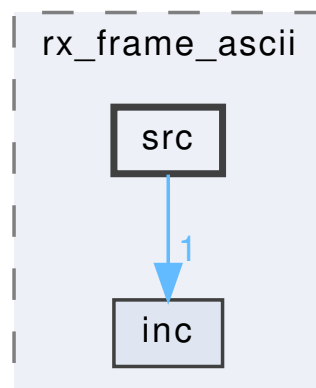


Directories

- directory [inc](#)
- directory [src](#)

5.4 libs/rx_frame_ascii/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_frame_ascii.c](#)
Frame ASCII Formatter Implementation.

Chapter 6

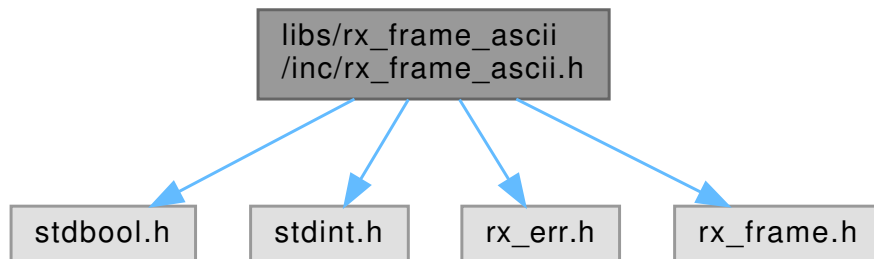
File Documentation

6.1 libs/rx_frame_ascii/inc/rx_frame_ascii.h File Reference

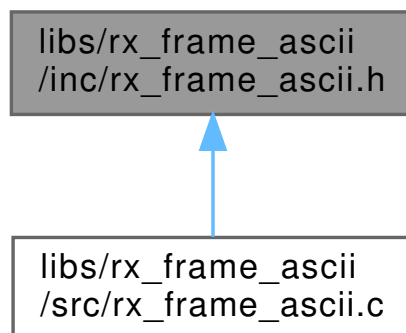
Frame ASCII Formatter for Human-Readable Frame Dumps.

```
#include <stdbool.h>
#include <stdint.h>
#include "rx_err.h"
#include "rx_frame.h"
```

Include dependency graph for rx_frame_ascii.h:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum `rx_frame_ascii_constants_t`: `uint16_t` { `k_frame_ascii_max_output` = 2048, `k_frame_ascii_hex_per_line` = 16, `k_frame_ascii_header_len` = 64, `k_frame_ascii_crc_len` = 32 }

ASCII formatter configuration constants.

Functions

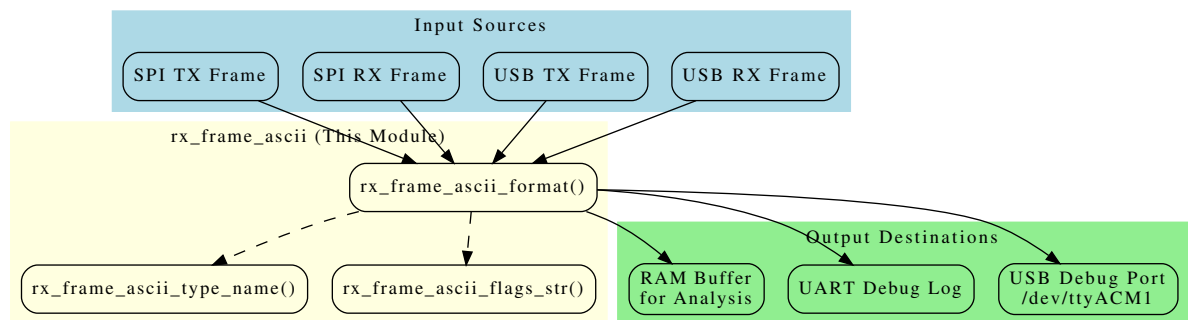
- `rx_err_t rx_frame_ascii_format` (const `rx_frame_t` *frame, bool is_tx, char *output, `uint32_t` output_max_len, `uint32_t` *output_len)
Format binary frame as human-readable ASCII.
- const char * `rx_frame_ascii_type_name` (`rx_frame_type_t` type)
Get frame type as human-readable string.
- const char * `rx_frame_ascii_flags_str` (`uint8_t` flags, char *buffer, `uint32_t` buffer_len)
Get frame flags as human-readable string.

6.1.1 Detailed Description

Frame ASCII Formatter for Human-Readable Frame Dumps.

Converts binary protocol frames into human-readable ASCII text for debugging and diagnostic output. This module provides a debug visualization layer that transforms the binary frame protocol (sync, headers, payload, CRC) into a structured text format suitable for terminal viewing or log analysis.

System Architecture



Output Format

The formatter produces structured text with three sections:

```
[RX] seq=1234 len=10 type=COMMAND flags=ACK|PRI
[0000] 48 65 6C 6F 20 55 53 42 21 00 00 00 00 00 00 Hello USB!.....
[CRC] 12345678
```

Section Details

Section	Description	Format
Header	Direction, sequence, length, type, flags	[TX/RX] seq=N len=N type=NAME flags=...

Section	Description	Format
Payload	Hex dump with ASCII sidebar	[NNNN] XX XX XX... ASCII...
CRC	32-bit CRC value	[CRC] XXXXXXXX

Performance Characteristics

Metric	Value	Notes
Format time (64B payload)	~50 us	At 240 MHz
Format time (2KB payload)	~500 us	At 240 MHz
Stack usage	~100 bytes	Fixed, no recursion
Output size ratio	~4:1	Output ~4x input size

Memory Usage

Component	Size	Description
Code size	~2 KB	Text section
Static data	~32 B	Hex lookup table
Stack (format)	~100 B	Per-call stack frame
Output buffer	User-provided	Typically 2048 bytes

Design Rationale

- **No dynamic allocation:** All buffers provided by caller (NASA Rule 3)
- **Bounded execution:** All loops have static upper bounds (NASA Rule 2)
- **No stdio dependency:** Custom formatters avoid printf overhead and non-reentrant issues
- **Hex + ASCII format:** Standard debugger-style output for easy payload inspection
- **Direction indicator:** TX/RX prefix distinguishes send vs receive for protocol tracing

Thread Safety

All functions are **thread-safe** and **reentrant**:

- No static mutable state
- All state passed via parameters
- Pure functions (same inputs -> same outputs)

Module Dependencies

- rx_frame.h: Frame structure definitions (rx_frame_t, rx_frame_header_t)
- rx_err.h: Error code definitions
- stdint.h: Fixed-width integer types
- stdbool.h: Boolean type

NASA Power of 10 Compliance

- Rule 1: [OK] No goto, setjmp, or recursion
- Rule 2: [OK] All loops bounded by payload length or constant
- Rule 3: [OK] No dynamic memory allocation
- Rule 4: [OK] Functions are concise (<60 lines)
- Rule 5: [OK] Input validation on all public functions
- Rule 6: [OK] Variables declared at smallest scope
- Rule 7: [OK] All return values checked
- Rule 8: [OK] Constants use C23 typed enums
- Rule 9: [OK] No function pointers (pure computation)
- Rule 10: [OK] Compiles with -Wall -Wextra -Werror

SOLID Principles

- **S (SRP)**: ASCII formatting only, no I/O operations
- **O (OCP)**: Flag/type tables easily extended without modifying logic
- **L (LSP)**: N/A (no inheritance hierarchy)
- **I (ISP)**: Three focused functions for different formatting needs
- **D (DIP)**: Depends only on frame abstractions, not concrete drivers

See also

[rx_frame.h](#) Frame structure definitions
[rx_usb.h](#) USB CDC output for debug port
[docs/sections/01_nanopb_protocol.tex](#) Protocol specification

Author

STAR Team

Date

2026-01-26

Copyright

Copyright (c) 2026 STAR Project. MIT License.

Since

Version 1.0.0

Definition in file [rx_frame_ascii.h](#).

6.1.2 Enumeration Type Documentation

6.1.2.1 rx_frame_ascii_constants_t

```
enum rx_frame_ascii_constants_t : uint16_t
```

ASCII formatter configuration constants.

Compile-time constants for buffer sizing, line formatting, and output structure. These values are tuned for typical debugging scenarios with protobuf message payloads up to 2KB.

Buffer Sizing Rationale

- Output is approximately 4x input size due to hex expansion + ASCII sidebar
- 2048 byte max output handles ~500 byte payloads comfortably
- 16 bytes per line provides standard hex dump format (same as xxd, hexdump)

Memory Impact

These constants only affect stack usage when output buffer is provided by caller. No static buffers are allocated based on these values.

Invariant

$$k_frame_ascii_max_output \geq 4 * k_frame_max_payload + \text{headers}$$

See also

[rx_frame_ascii_format\(\)](#) Uses these constants for buffer validation

Since

Version 1.0.0

Enumerator

k_frame_ascii_max_output	Maximum recommended output buffer size in bytes. Recommended minimum buffer size for rx_frame_ascii_format() to handle maximum-size frames without truncation. Accounts for header line, hex dump lines (4 chars per byte + ASCII sidebar), and CRC footer. Value: 2048 bytes Calculation: ~500 payload x 4 expansion + 64 header + 32 CRC
--------------------------	---

k_frame_ascii_hex_per_line	Bytes per line in hex payload dump. Standard hex dump format with 16 bytes per line. Produces output like: [0000] 00 01 02 ... 0F Value: 16 bytes (standard format) Note Matches xxd and hexdump default format
k_frame_ascii_header_len	Maximum header line length in characters. Buffer size for the header metadata line containing direction, sequence, length, type, and flags. Example: [RX] seq=65535 len=2048 type=COMMAND flags=ACK PRI RETX Value: 64 characters
k_frame_ascii_crc_len	Maximum CRC line length in characters. Buffer size for the CRC footer line. Format: [CRC] XXXXXXXX Value: 32 characters

Definition at line 179 of file `rx_frame_ascii.h`.

```

00179         : uint16_t {
00180     /**
00181     * @brief Maximum recommended output buffer size in bytes
00182     * @details
00183     * Recommended minimum buffer size for rx_frame_ascii_format() to handle
00184     * maximum-size frames without truncation. Accounts for header line, hex
00185     * dump lines (4 chars per byte + ASCII sidebar), and CRC footer.
00186     * @par Value: 2048 bytes
00187     * @par Calculation: ~500 payload x 4 expansion + 64 header + 32 CRC
00188     */
00189     k_frame_ascii_max_output = 2048,
00190
00191     /**
00192     * @brief Bytes per line in hex payload dump
00193     * @details
00194     * Standard hex dump format with 16 bytes per line. Produces output like:
00195     * `[0000] 00 01 02 ... 0F' .....
00196     * @par Value: 16 bytes (standard format)
00197     * @note Matches xxd and hexdump default format
00198     */
00199     k_frame_ascii_hex_per_line = 16,
00200
00201     /**
00202     * @brief Maximum header line length in characters
00203     * @details
00204     * Buffer size for the header metadata line containing direction, sequence,
00205     * length, type, and flags. Example: `[RX] seq=65535 len=2048 type=COMMAND flags=ACK|PRI|RETX'
00206     * @par Value: 64 characters
00207     */
00208     k_frame_ascii_header_len = 64,
00209
00210     /**
00211     * @brief Maximum CRC line length in characters
00212     * @details
00213     * Buffer size for the CRC footer line. Format: `[CRC] XXXXXXXX'
00214     * @par Value: 32 characters
00215     */
00216     k_frame_ascii_crc_len = 32,
00217 } rx_frame_ascii_constants_t;

```

6.1.3 Function Documentation

6.1.3.1 rx_frame_ascii_flags_str()

```
const char * rx_frame_ascii_flags_str (
    uint8_t flags,
    char * buffer,
    uint32_t buffer_len)
```

Get frame flags as human-readable string.

Formats a flags bitmask into a compact pipe-separated string showing all active flags. If no flags are set (value == 0), returns "NONE".

Flag Mapping

Flag Bit	String	Description
k_frame_flag_requires_ack (0x01)	"ACK"	Acknowledgment required
k_frame_flag_retransmit (0x02)	"RETX"	Retransmission
k_frame_flag_priority (0x04)	"PRI"	High priority
k_frame_flag_fec_enabled (0x08)	"FEC"	FEC encoding
k_frame_flag_soft_nack (0x10)	"SOFT"	Soft NACK
k_frame_flag_none (0x00)	"NONE"	No flags set

Output Examples

Input Flags	Output String
0x00	"NONE"
0x01	"ACK"

| 0x03 | "ACK|RETX" | | 0x07 | "ACK|RETX|PRI" | | 0x1F | "ACK|RETX|PRI|FEC|SOFT" |

Parameters

in	<i>flags</i>	Frame flags bitmask - Value from rx_frame_flags_t or combination thereof 0 produces "NONE"
----	--------------	---

out	<i>buffer</i>	Output buffer for formatted string • Must be valid non-nullptr • Caller maintains ownership • Will be null-terminated
in	<i>buffer_len</i>	Size of output buffer in bytes • Minimum recommended: 32 bytes • Must be > 0

Returns

Pointer to buffer with formatted flags string

Return values

<i>buffer</i>	On success (contains formatted string)
<code>""</code>	(empty string) If buffer is nullptr or buffer_len is 0

Precondition

`buffer != nullptr` (or returns empty string)

`buffer_len > 0` (or returns empty string)

Postcondition

buffer is null-terminated (if valid)

Return value == buffer (if valid)

Note

Thread-safe: output written to caller-provided buffer

Reentrant: no static mutable state

Warning

Buffer must be large enough for all flags + separators

Returns empty string literal `""` if buffer is nullptr

Basic Usage Example

```
char flags_buf[32];
uint8_t flags = k_frame_flag_requires_ack | k_frame_flag_priority;

const char* str = rx_frame_ascii_flags_str(flags, flags_buf, sizeof(flags_buf));
// str == "ACK|PRI"
// flags_buf contains "ACK|PRI\0"
```


Error Handling Example

```
// NULL buffer returns empty string
const char* empty = rx_frame_ascii_flags_str(0x01, nullptr, 0);
// empty == ""

// Zero-length buffer returns empty string
char tiny[1];
const char* also_empty = rx_frame_ascii_flags_str(0x01, tiny, 0);
// also_empty == ""
```

Performance

- Execution time: $O(n)$ where n = number of flag bits checked
- Stack usage: ~ 10 bytes

See also

[rx_frame_flags_t](#) Flag definitions

[rx_frame_ascii_format\(\)](#) Uses this function internally

Since

Version 1.0.0

Get frame flags as human-readable string.

Implementation of [rx_frame_ascii_flags_str\(\)](#). Iterates through known flag bits, appending names with pipe separators.

Parameters

in	<i>flags</i>	Frame flags bitmask
out	<i>buffer</i>	Output buffer for formatted string
in	<i>buffer_len</i>	Size of output buffer in bytes

Returns

Pointer to buffer (or "" if buffer is nullptr)

Note

Flags formatted as pipe-separated: "ACK|RETX|PRI"

See also

[rx_frame_ascii.h](#) for full documentation

Definition at line 874 of file [rx_frame_ascii.c](#).

```

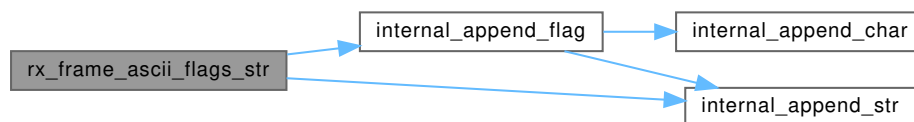
00875 {
00876     if (buffer == nullptr || buffer_len == k_empty_buffer) {
00877         return "";
00878     }
00879
00880     uint32_t pos = 0;
00881
00882     if (flags == k_frame_flag_none) {
00883         pos = internal_append_str(buffer, pos, buffer_len, "NONE");
00884         buffer[pos] = '\0';
00885         return buffer;
00886     }
00887
00888     bool first = true;
00889
00890     /* Use helper to append each flag */
00891     pos =
00892         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_requires_ack, "ACK");
00893     pos =
00894         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_retransmit, "RETX");
00895     pos = internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_priority, "PRI");
00896     pos =
00897         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_fec_enabled, "FEC");
00898     pos =
00899         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_soft_nack, "SOFT");
00900
00901     buffer[pos] = '\0';
00902     return buffer;
00903 }

```

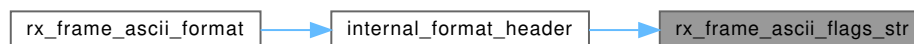
References [internal_append_flag\(\)](#), [internal_append_str\(\)](#), and [k_empty_buffer](#).

Referenced by [internal_format_header\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.2 rx_frame_ascii_format()

```

rx_err_t rx_frame_ascii_format (
    const rx_frame_t * frame,
    bool is_tx,
    char * output,
    uint32_t output_max_len,
    uint32_t * output_len) [nodiscard]

```

Format binary frame as human-readable ASCII.

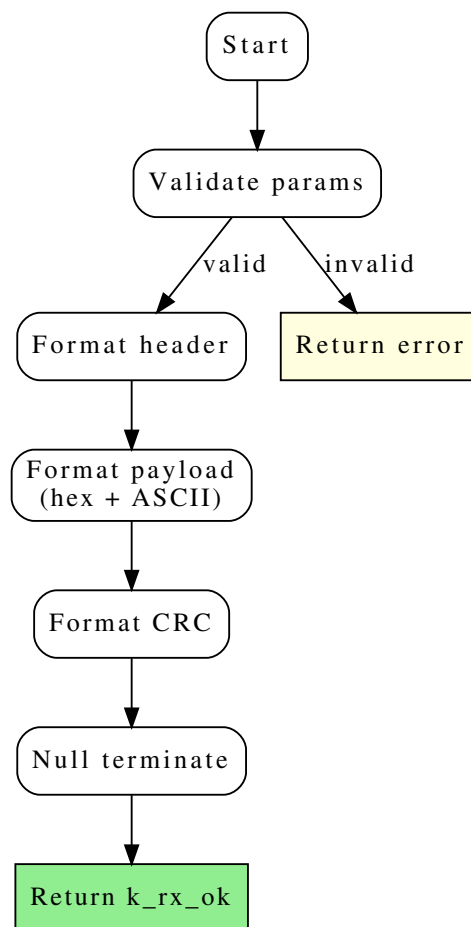
Converts a complete frame structure into a formatted ASCII string suitable for debugging output, logging, or terminal display. The output is structured in three sections: header metadata, hex payload dump with ASCII sidebar, and CRC footer.

Algorithm Steps

1. Validate all input pointers and buffer size
2. Format header line: direction, sequence, length, type, flags
3. Format payload as hex dump with 16 bytes per line + ASCII sidebar
4. Format CRC footer with 8-digit hex value
5. Null-terminate output and return length

Output Structure

```
[TX] seq=1234 len=10 type=COMMAND flags=ACK|PRI
[0000] 48 65 6C 6C 6F 20 55 53 42 21 00 00 00 00 00 00 Hello USB!.....
[CRC] 12345678
```



Parameters

in	<i>frame</i>	Frame to format • Must be valid non-nullptr • header.length must be $\leq$ k_frame_max_payload • payload data must be valid for header.length bytes
----	--------------	--

in	<i>is_tx</i>	Direction indicator • true: Prefixes output with "[TX] " (transmit direction) • false: Prefixes output with "[RX] " (receive direction)
out	<i>output</i>	Output buffer for ASCII text • Must be valid non-nullptr • Caller maintains ownership • Will be null-terminated on success
in	<i>output_max_len</i>	Maximum bytes to write to output buffer • Must be ≥ 64 (minimum for header + CRC with empty payload) • Recommended: <code>k_frame_ascii_max_output</code> (2048) • Larger payloads require proportionally larger buffers
out	<i>output_len</i>	Actual bytes written (excluding null terminator) • Must be valid non-nullptr • Set only on success

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Frame formatted successfully
<i>k_rx_err_invalid_arg</i>	frame, output, or output_len is nullptr, or frame->header.length > k_frame_max_payload
<i>k_rx_err_no_mem</i>	Output buffer too small (< 64 bytes or truncation occurred)

Precondition

```

frame != nullptr
output != nullptr
output_len != nullptr
output_max_len >= 64
frame->header.length <= k_frame_max_payload

```

Postcondition

```

output is null-terminated
*output_len contains actual output length (on success)
output buffer contains formatted ASCII text (on success)

```

Invariant

Output never exceeds output_max_len bytes

Note

Thread-safe: no static mutable state

Reentrant: all state passed via parameters

Warning

Truncation returns k_rx_err_no_mem; output may be partial

Empty payload (length=0) produces [PAYLOAD] (empty) line

Basic Usage Example

```
rx_frame_t frame;
char output[2048];
uint32_t output_len;

// Receive a frame from USB or SPI
rx_usb_comm_receive(&handle, &frame, 100);

// Format for debug output
rx_err_t err = rx_frame_ascii_format(&frame, false, output, sizeof(output), &output_len);
if (err == k_rx_ok) {
    // Send to debug USB port
    rx_usb_write(k_usb_port_debug, (const uint8_t*)output, output_len);
}
```

Error Handling Example

```
rx_err_t err = rx_frame_ascii_format(&frame, true, buf, sizeof(buf), &len);
switch (err) {
    case k_rx_ok:
        // Success - output contains formatted frame
        break;
    case k_rx_err_invalid_arg:
        rx_log_error("ASCII", "Invalid frame or buffer pointer");
        break;
    case k_rx_err_no_mem:
        rx_log_warn("ASCII", "Buffer too small, output truncated");
        break;
    default:
        rx_log_error("ASCII", "Unexpected error");
        break;
}
```

Performance

- Execution time: ~50 us for 64-byte payload @ 240 MHz
- Stack usage: ~100 bytes (fixed)
- No heap allocation

See also

[rx_frame_ascii_type_name\(\)](#) Convert type enum to string

[rx_frame_ascii_flags_str\(\)](#) Convert flags to string

[rx_frame_t](#) Frame structure definition

Since

Version 1.0.0

Format binary frame as human-readable ASCII.

Implementation of [rx_frame_ascii_format\(\)](#). Orchestrates header, payload, and CRC formatting using internal helper functions.

Implementation Notes

- Validates all inputs before any output
- Calls [internal_format_header\(\)](#) for metadata line
- Calls [internal_format_payload\(\)](#) for hex dump
- Appends CRC footer directly
- Detects truncation and returns `k_rx_err_no_mem`

Parameters

in	<i>frame</i>	Frame to format (must not be NULL)
in	<i>is_tx</i>	Direction (true=TX, false=RX)
out	<i>output</i>	Output buffer for formatted string
in	<i>output_max_len</i>	Maximum output buffer size
out	<i>output_len</i>	Pointer to receive actual output length

Returns

`k_rx_ok` on success
`k_rx_err_invalid_arg` if frame/output/output_len is nullptr
`k_rx_err_no_mem` if output buffer is too small

See also

[rx_frame_ascii.h](#) for full documentation

Definition at line 931 of file [rx_frame_ascii.c](#).

```

00936 {
00937     /* Rule 5: Pre-condition validation */
00938     if (frame == nullptr) {
00939         return k_rx_err_invalid_arg;
00940     }
00941     if (output == nullptr) {
00942         return k_rx_err_invalid_arg;
00943     }
00944     if (output_len == nullptr) {
00945         return k_rx_err_invalid_arg;
00946     }
00947     if (output_max_len < k_min_output_size) {
00948         return k_rx_err_no_mem;
00949     }
00950
00951     /* Validate payload length does not exceed frame capacity */
00952     if (frame->header.length > k_frame_max_payload) {
00953         return k_rx_err_invalid_arg;
00954     }
00955
00956     uint32_t pos = 0;
00957

```

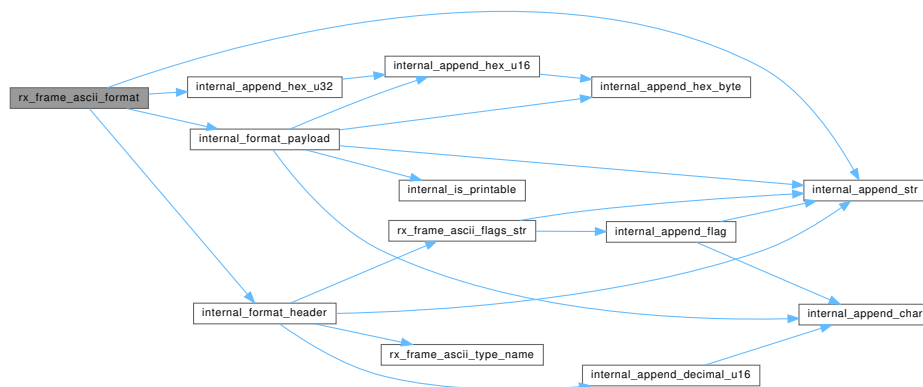
```

00958  /* Format header metadata */
00959  pos = internal_format_header(output, pos, output_max_len, &frame->header, is_tx);
00960
00961  /* Payload hex dump - use validated length to prevent out-of-bounds reads */
00962  pos = internal_format_payload(output, pos, output_max_len, frame->payload, frame->header.length);
00963
00964  /* CRC */
00965  pos = internal_append_str(output, pos, output_max_len, "[CRC] ");
00966  pos = internal_append_hex_u32(output, pos, output_max_len, frame->crc);
00967  pos = internal_append_str(output, pos, output_max_len, "\r\n");
00968
00969  /* Rule 5: Post-condition validation - check for truncation */
00970  /* If pos >= output_max_len, we don't have room for null terminator */
00971  if (pos >= output_max_len) {
00972      /* Buffer was too small - data was truncated */
00973      if (output_max_len > 0) {
00974          output[output_max_len - 1] = '\0'; /* Null terminate at buffer end */
00975      }
00976      return k_rx_err_no_mem;
00977  }
00978
00979  /* Null terminate */
00980  output[pos] = '\0';
00981
00982  *output_len = pos;
00983
00984  return k_rx_ok;
00985 }

```

References [internal_append_hex_u32\(\)](#), [internal_append_str\(\)](#), [internal_format_header\(\)](#), [internal_format_payload\(\)](#), and [k_min_output_size](#).

Here is the call graph for this function:



6.1.3.3 rx_frame_ascii_type_name()

```

const char * rx_frame_ascii_type_name (
    rx_frame_type_t type)

```

Get frame type as human-readable string.

Converts a frame type enumeration value to its corresponding string name for display or logging. Returns a static string literal that does not require freeing.

Type Mapping

Type Value	String Output
k_frame_type_ping (0x00)	"PING"

Type Value	String Output
k_frame_type_pong (0x01)	"PONG"
k_frame_type_command (0x10)	"COMMAND"
k_frame_type_response (0x11)	"RESPONSE"
k_frame_type_ack (0x12)	"ACK"
k_frame_type_nack (0x13)	"NACK"
k_frame_type_reset_ack (0xFE)	"RESET_ACK"
k_frame_type_reset (0xFF)	"RESET"
(any other value)	"UNKNOWN"

Parameters

in	<i>type</i>	Frame type enumeration value - Any rx_frame_type_t value - Unknown values return "UNKNOWN"
----	-------------	---

Returns

Pointer to static string literal

Return values

<i>PING</i>	For k_frame_type_ping
<i>PONG</i>	For k_frame_type_pong
<i>COMMAND</i>	For k_frame_type_command
<i>RESPONSE</i>	For k_frame_type_response
<i>ACK</i>	For k_frame_type_ack
<i>NACK</i>	For k_frame_type_nack
<i>RESET_ACK</i>	For k_frame_type_reset_ack
<i>RESET</i>	For k_frame_type_reset
<i>UNKNOWN</i>	For unrecognized values

Precondition

None (pure function, always valid)

Postcondition

Return value is non-nullptr to null-terminated string

Return value points to static read-only memory

Note

Thread-safe: returns pointer to static const data

Reentrant: no mutable state

Do NOT free the returned pointer

Example

```
rx_frame_type_t type = k_frame_type_command;
const char* name = rx_frame_ascii_type_name(type);
// name == "COMMAND"

// Safe with invalid values
const char* unknown = rx_frame_ascii_type_name((rx_frame_type_t)0xFF);
// unknown == "UNKNOWN"
```

Performance

- Execution time: O(1) - switch statement
- Stack usage: 0 bytes additional

See also

[rx_frame_type_t](#) Frame type enumeration

[rx_frame_ascii_format\(\)](#) Uses this function internally

Since

Version 1.0.0

Get frame type as human-readable string.

Implementation of [rx_frame_ascii_type_name\(\)](#). Uses switch statement for O(1) lookup with compile-time string literals.

Parameters

in	<i>type</i>	Frame type enum value
----	-------------	-----------------------

Returns

String representation ("PING", "PONG", "COMMAND", "RESPONSE", "ACK", "NACK", "RESET_ACK", "RESET", or "UNKNOWN")

See also

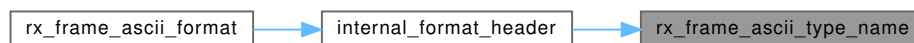
[rx_frame_ascii.h](#) for full documentation

Definition at line 776 of file [rx_frame_ascii.c](#).

```
00777 {
00778     switch (type) {
00779         case k_frame_type_ping:
00780             return "PING";
00781         case k_frame_type_pong:
00782             return "PONG";
00783         case k_frame_type_command:
00784             return "COMMAND";
00785         case k_frame_type_response:
00786             return "RESPONSE";
00787         case k_frame_type_ack:
00788             return "ACK";
00789         case k_frame_type_nack:
00790             return "NACK";
00791         case k_frame_type_reset_ack:
00792             return "RESET_ACK";
00793         case k_frame_type_reset:
00794             return "RESET";
00795         default:
00796             return "UNKNOWN";
00797     }
00798 }
```

Referenced by [internal_format_header\(\)](#).

Here is the caller graph for this function:



6.2 rx_frame_ascii.h

[Go to the documentation of this file.](#)

```
00001 /* lib/rx_frame_ascii/inc/rx_frame_ascii.h */
00002
00003 /**
00004  * @file rx_frame_ascii.h
00005  * @brief Frame ASCII Formatter for Human-Readable Frame Dumps
00006  *
00007  * @details
00008  * Converts binary protocol frames into human-readable ASCII text for debugging
00009  * and diagnostic output. This module provides a debug visualization layer that
00010  * transforms the binary frame protocol (sync, headers, payload, CRC) into a
00011  * structured text format suitable for terminal viewing or log analysis.
00012  *
00013  * @par System Architecture
00014  * @dot
00015  * digraph frame_ascii_architecture {
00016  *     rankdir=TB;
00017  *     node [shape=box, style=rounded];
00018  *
00019  *     subgraph cluster_input {
00020  *         label="Input Sources";
00021  *         style=filled;
00022  *         color=lightblue;
00023  *         usb_rx [label="USB RX Frame"];
00024  *         usb_tx [label="USB TX Frame"];
00025  *         spi_rx [label="SPI RX Frame"];
00026  *         spi_tx [label="SPI TX Frame"];
00027  *     }
00028  *
00029  *     subgraph cluster_formatter {
00030  *         label="rx_frame_ascii (This Module)";
00031  *         style=filled;
00032  *         color=lightyellow;
00033  *         format [label="rx_frame_ascii_format()"];
00034  *         type_name [label="rx_frame_ascii_type_name()"];
00035  *         flags_str [label="rx_frame_ascii_flags_str()"];
00036  *     }
00037  }
```

```

00037 *
00038 * subgraph cluster_output {
00039 *   label="Output Destinations";
00040 *   style=filled;
00041 *   color=lightgreen;
00042 *   usb_debug [label="USB Debug Port\n/dev/ttyACM1"];
00043 *   uart_log [label="UART Debug Log"];
00044 *   buffer [label="RAM Buffer\nfor Analysis"];
00045 * }
00046 *
00047 * usb_rx -> format;
00048 * usb_tx -> format;
00049 * spi_rx -> format;
00050 * spi_tx -> format;
00051 * format -> usb_debug;
00052 * format -> uart_log;
00053 * format -> buffer;
00054 * format -> type_name [style=dashed];
00055 * format -> flags_str [style=dashed];
00056 * }
00057 * @enddot
00058 *
00059 * @par Output Format
00060 * The formatter produces structured text with three sections:
00061 * ```
00062 * [RX] seq=1234 len=10 type=COMMAND flags=ACK|PRI
00063 * [0000] 48 65 6C 6F 20 55 53 42 21 00 00 00 00 00 00 Hello USB!.....
00064 * [CRC] 12345678
00065 * ```
00066 *
00067 * @par Section Details
00068 * | Section | Description | Format |
00069 * |-----|-----|-----|
00070 * | Header | Direction, sequence, length, type, flags | `[TX/RX] seq=N len=N type=NAME flags=...` |
00071 * | Payload | Hex dump with ASCII sidebar | `[NNNN] XX XX XX... ASCII...` |
00072 * | CRC | 32-bit CRC value | `[CRC] XXXXXXXXX` |
00073 *
00074 * @par Performance Characteristics
00075 * | Metric | Value | Notes |
00076 * |-----|-----|-----|
00077 * | Format time (64B payload) | ~50 us | At 240 MHz |
00078 * | Format time (2KB payload) | ~500 us | At 240 MHz |
00079 * | Stack usage | ~100 bytes | Fixed, no recursion |
00080 * | Output size ratio | ~4:1 | Output ~4x input size |
00081 *
00082 * @par Memory Usage
00083 * | Component | Size | Description |
00084 * |-----|-----|-----|
00085 * | Code size | ~2 KB | Text section |
00086 * | Static data | ~32 B | Hex lookup table |
00087 * | Stack (format) | ~100 B | Per-call stack frame |
00088 * | Output buffer | User-provided | Typically 2048 bytes |
00089 *
00090 * @par Design Rationale
00091 * - **No dynamic allocation**: All buffers provided by caller (NASA Rule 3)
00092 * - **Bounded execution**: All loops have static upper bounds (NASA Rule 2)
00093 * - **No stdio dependency**: Custom formatters avoid printf overhead and non-reentrant issues
00094 * - **Hex + ASCII format**: Standard debugger-style output for easy payload inspection
00095 * - **Direction indicator**: TX/RX prefix distinguishes send vs receive for protocol tracing
00096 *
00097 * @par Thread Safety
00098 * All functions are **thread-safe** and **reentrant**:
00099 * - No static mutable state
00100 * - All state passed via parameters
00101 * - Pure functions (same inputs -> same outputs)
00102 *
00103 * @par Module Dependencies
00104 * - rx_frame.h: Frame structure definitions (rx_frame_t, rx_frame_header_t)
00105 * - rx_err.h: Error code definitions
00106 * - stdint.h: Fixed-width integer types
00107 * - stdbool.h: Boolean type
00108 *
00109 * @par NASA Power of 10 Compliance
00110 * - Rule 1: [OK] No goto, setjmp, or recursion
00111 * - Rule 2: [OK] All loops bounded by payload length or constant
00112 * - Rule 3: [OK] No dynamic memory allocation
00113 * - Rule 4: [OK] Functions are concise (<60 lines)
00114 * - Rule 5: [OK] Input validation on all public functions
00115 * - Rule 6: [OK] Variables declared at smallest scope
00116 * - Rule 7: [OK] All return values checked
00117 * - Rule 8: [OK] Constants use C23 typed enums
00118 * - Rule 9: [OK] No function pointers (pure computation)
00119 * - Rule 10: [OK] Compiles with -Wall -Wextra -Werror
00120 *
00121 * @par SOLID Principles
00122 * - **S (SRP)**: ASCII formatting only, no I/O operations
00123 * - **O (OCP)**: Flag/type tables easily extended without modifying logic

```

```

00124 * - **L (LSP):** N/A (no inheritance hierarchy)
00125 * - **I (ISP):** Three focused functions for different formatting needs
00126 * - **D (DIP):** Depends only on frame abstractions, not concrete drivers
00127 *
00128 * @see rx_frame.h Frame structure definitions
00129 * @see rx_usb.h USB CDC output for debug port
00130 * @see docs/sections/01_nanopb_protocol.tex Protocol specification
00131 *
00132 * @author STAR Team
00133 * @date 2026-01-26
00134 * @copyright Copyright (c) 2026 STAR Project. MIT License.
00135 * @since Version 1.0.0
00136 */
00137
00138 #pragma once
00139
00140 #include <stdbool.h>
00141 #include <stdint.h>
00142
00143 #include "rx_err.h"
00144 #include "rx_frame.h"
00145
00146 #ifdef __cplusplus
00147 extern "C" {
00148 #endif
00149
00150 /*
=====
00151 * Configuration Constants
00152 *
=====
00153 */
00154 /**
00155  * @enum rx_frame_ascii_constants_t
00156  * @brief ASCII formatter configuration constants
00157  *
00158  * @details
00159  * Compile-time constants for buffer sizing, line formatting, and output
00160  * structure. These values are tuned for typical debugging scenarios with
00161  * protobuf message payloads up to 2KB.
00162  *
00163  * @par Buffer Sizing Rationale
00164  * - Output is approximately 4x input size due to hex expansion + ASCII sidebar
00165  * - 2048 byte max output handles ~500 byte payloads comfortably
00166  * - 16 bytes per line provides standard hex dump format (same as xxd, hexdump)
00167  *
00168  * @par Memory Impact
00169  * These constants only affect stack usage when output buffer is provided by
00170  * caller. No static buffers are allocated based on these values.
00171  *
00172  * @invariant k_frame_ascii_max_output >= 4 * k_frame_max_payload + headers
00173  *
00174  * @see rx_frame_ascii_format() Uses these constants for buffer validation
00175  *
00176  * @since Version 1.0.0
00177 */
00178 typedef enum : uint16_t {
00179 /**
00180  * @brief Maximum recommended output buffer size in bytes
00181  * @details
00182  * Recommended minimum buffer size for rx_frame_ascii_format() to handle
00183  * maximum-size frames without truncation. Accounts for header line, hex
00184  * dump lines (4 chars per byte + ASCII sidebar), and CRC footer.
00185  * @par Value: 2048 bytes
00186  * @par Calculation: ~500 payload x 4 expansion + 64 header + 32 CRC
00187  */
00188 k_frame_ascii_max_output = 2048,
00189
00190 /**
00191  * @brief Bytes per line in hex payload dump
00192  * @details
00193  * Standard hex dump format with 16 bytes per line. Produces output like:
00194  * `[0000] 00 01 02 ... 0F .....`
00195  * @par Value: 16 bytes (standard format)
00196  * @note Matches xxd and hexdump default format
00197  */
00198 k_frame_ascii_hex_per_line = 16,
00199
00200 /**
00201  * @brief Maximum header line length in characters
00202  * @details
00203  * Buffer size for the header metadata line containing direction, sequence,
00204  * length, type, and flags. Example: `[RX] seq=65535 len=2048 type=COMMAND flags=ACK|PRI|RETX`
00205  * @par Value: 64 characters
00206  */
00207 k_frame_ascii_header_len = 64,
00208

```

```

00209
00210 /**
00211  * @brief Maximum CRC line length in characters
00212  * @details
00213  * Buffer size for the CRC footer line. Format: `[CRC] XXXXXXXX`
00214  * @par Value: 32 characters
00215  */
00216 k_frame_ascii_crc_len = 32,
00217 } rx_frame_ascii_constants_t;
00218
00219 /*
=====
00220 * Public API
00221 *
=====
00222 */
00223
00224 /**
00225  * @brief Format binary frame as human-readable ASCII
00226  * @details
00227  * Converts a complete frame structure into a formatted ASCII string suitable
00228  * for debugging output, logging, or terminal display. The output is structured
00229  * in three sections: header metadata, hex payload dump with ASCII sidebar, and
00230  * CRC footer.
00231  * @par Algorithm Steps
00232  * 1. Validate all input pointers and buffer size
00233  * 2. Format header line: direction, sequence, length, type, flags
00234  * 3. Format payload as hex dump with 16 bytes per line + ASCII sidebar
00235  * 4. Format CRC footer with 8-digit hex value
00236  * 5. Null-terminate output and return length
00237
00238  * @par Output Structure
00239  * ```
00240  * [TX] seq=1234 len=10 type=COMMAND flags=ACK|PRI
00241  * [0000] 48 65 6C 6C 6F 20 55 53 42 21 00 00 00 00 00 00 Hello USB!.....
00242  * [CRC] 12345678
00243  * ```
00244  * @dot
00245  * digraph format_flow {
00246  *   rankdir=TB;
00247  *   node [shape=box, style=rounded];
00248  *   start [label="Start"];
00249  *   validate [label="Validate params"];
00250  *   header [label="Format header"];
00251  *   payload [label="Format payload\n(hex + ASCII)"];
00252  *   crc [label="Format CRC"];
00253  *   terminate [label="Null terminate"];
00254  *   done [label="Return k_rx_ok" style=filled, fillcolor=lightgreen];
00255  *   error [label="Return error" style=filled, fillcolor=lightyellow];
00256  *   start -> validate;
00257  *   validate -> header [label="valid"];
00258  *   validate -> error [label="invalid"];
00259  *   header -> payload;
00260  *   payload -> crc;
00261  *   crc -> terminate;
00262  *   terminate -> done;
00263  * }
00264  * @enddot
00265
00266  * @param[in] frame Frame to format
00267  * - Must be valid non-nullptr
00268  * - header.length must be <= k_frame_max_payload
00269  * - payload data must be valid for header.length bytes
00270  * @param[in] is_tx Direction indicator
00271  * - true: Prefixes output with "[TX]" (transmit direction)
00272  * - false: Prefixes output with "[RX]" (receive direction)
00273  * @param[out] output Output buffer for ASCII text
00274  * - Must be valid non-nullptr
00275  * - Caller maintains ownership
00276  * - Will be null-terminated on success
00277  * @param[in] output_max_len Maximum bytes to write to output buffer
00278  * - Must be >= 64 (minimum for header + CRC with empty payload)
00279  * - Recommended: k_frame_ascii_max_output (2048)
00280  * - Larger payloads require proportionally larger buffers
00281  * @param[out] output_len Actual bytes written (excluding null terminator)
00282  * - Must be valid non-nullptr
00283  * - Set only on success
00284
00285  * @return rx_err_t Error code
00286  * @retval k_rx_ok Frame formatted successfully
00287  * @retval k_rx_err_invalid_arg frame, output, or output_len is nullptr,
00288  * or frame->header.length > k_frame_max_payload
00289  * @retval k_rx_err_no_mem Output buffer too small (< 64 bytes or truncation occurred)
00290
00291
00292
00293

```

```

00294 * @pre frame != nullptr
00295 * @pre output != nullptr
00296 * @pre output_len != nullptr
00297 * @pre output_max_len >= 64
00298 * @pre frame->header.length <= k_frame_max_payload
00299 *
00300 * @post output is null-terminated
00301 * @post *output_len contains actual output length (on success)
00302 * @post output buffer contains formatted ASCII text (on success)
00303 *
00304 * @invariant Output never exceeds output_max_len bytes
00305 *
00306 * @note Thread-safe: no static mutable state
00307 * @note Reentrant: all state passed via parameters
00308 *
00309 * @warning Truncation returns k_rx_err_no_mem; output may be partial
00310 * @warning Empty payload (length=0) produces `[PAYLOAD] (empty)' line
00311 *
00312 * @par Basic Usage Example
00313 * @code
00314 * rx_frame_t frame;
00315 * char output[2048];
00316 * uint32_t output_len;
00317 *
00318 * // Receive a frame from USB or SPI
00319 * rx_usb_comm_receive(&handle, &frame, 100);
00320 *
00321 * // Format for debug output
00322 * rx_err_t err = rx_frame_ascii_format(&frame, false, output, sizeof(output), &output_len);
00323 * if (err == k_rx_ok) {
00324 *     // Send to debug USB port
00325 *     rx_usb_write(k_usb_port_debug, (const uint8_t*)output, output_len);
00326 * }
00327 * @endcode
00328 *
00329 * @par Error Handling Example
00330 * @code
00331 * rx_err_t err = rx_frame_ascii_format(&frame, true, buf, sizeof(buf), &len);
00332 * switch (err) {
00333 *     case k_rx_ok:
00334 *         // Success - output contains formatted frame
00335 *         break;
00336 *     case k_rx_err_invalid_arg:
00337 *         rx_log_error("ASCII", "Invalid frame or buffer pointer");
00338 *         break;
00339 *     case k_rx_err_no_mem:
00340 *         rx_log_warn("ASCII", "Buffer too small, output truncated");
00341 *         break;
00342 *     default:
00343 *         rx_log_error("ASCII", "Unexpected error");
00344 *         break;
00345 * }
00346 * @endcode
00347 *
00348 * @par Performance
00349 * - Execution time: ~50 us for 64-byte payload @ 240 MHz
00350 * - Stack usage: ~100 bytes (fixed)
00351 * - No heap allocation
00352 *
00353 * @see rx_frame_ascii_type_name() Convert type enum to string
00354 * @see rx_frame_ascii_flags_str() Convert flags to string
00355 * @see rx_frame_t Frame structure definition
00356 *
00357 * @since Version 1.0.0
00358 */
00359 [[nodiscard]] rx_err_t rx_frame_ascii_format(const rx_frame_t* frame,
00360                                             bool is_tx,
00361                                             char* output,
00362                                             uint32_t output_max_len,
00363                                             uint32_t* output_len);
00364
00365 /**
00366 * @brief Get frame type as human-readable string
00367 *
00368 * @details
00369 * Converts a frame type enumeration value to its corresponding string name
00370 * for display or logging. Returns a static string literal that does not
00371 * require freeing.
00372 *
00373 * @par Type Mapping
00374 * | Type Value | String Output |
00375 * |-----|-----|
00376 * | k_frame_type_ping (0x00) | "PING" |
00377 * | k_frame_type_pong (0x01) | "PONG" |
00378 * | k_frame_type_command (0x10) | "COMMAND" |
00379 * | k_frame_type_response (0x11) | "RESPONSE" |
00380 * | k_frame_type_ack (0x12) | "ACK" |

```

```

00381 * | k_frame_type_nack (0x13) | "NACK" |
00382 * | k_frame_type_reset_ack (0xFE) | "RESET_ACK" |
00383 * | k_frame_type_reset (0xFF) | "RESET" |
00384 * | (any other value) | "UNKNOWN" |
00385 *
00386 * @param[in] type Frame type enumeration value
00387 * - Any rx_frame_type_t value
00388 * - Unknown values return "UNKNOWN"
00389 *
00390 * @return Pointer to static string literal
00391 * @retval "PING" For k_frame_type_ping
00392 * @retval "PONG" For k_frame_type_pong
00393 * @retval "COMMAND" For k_frame_type_command
00394 * @retval "RESPONSE" For k_frame_type_response
00395 * @retval "ACK" For k_frame_type_ack
00396 * @retval "NACK" For k_frame_type_nack
00397 * @retval "RESET_ACK" For k_frame_type_reset_ack
00398 * @retval "RESET" For k_frame_type_reset
00399 * @retval "UNKNOWN" For unrecognized values
00400 *
00401 * @pre None (pure function, always valid)
00402 *
00403 * @post Return value is non-nullptr to null-terminated string
00404 * @post Return value points to static read-only memory
00405 *
00406 * @note Thread-safe: returns pointer to static const data
00407 * @note Reentrant: no mutable state
00408 * @note Do NOT free the returned pointer
00409 *
00410 * @par Example
00411 * @code
00412 * rx_frame_type_t type = k_frame_type_command;
00413 * const char* name = rx_frame_ascii_type_name(type);
00414 * // name == "COMMAND"
00415 *
00416 * // Safe with invalid values
00417 * const char* unknown = rx_frame_ascii_type_name((rx_frame_type_t)0xFF);
00418 * // unknown == "UNKNOWN"
00419 * @endcode
00420 *
00421 * @par Performance
00422 * - Execution time: O(1) - switch statement
00423 * - Stack usage: 0 bytes additional
00424 *
00425 * @see rx_frame_type_t Frame type enumeration
00426 * @see rx_frame_ascii_format() Uses this function internally
00427 *
00428 * @since Version 1.0.0
00429 */
00430 const char* rx_frame_ascii_type_name(rx_frame_type_t type);
00431
00432 /**
00433 * @brief Get frame flags as human-readable string
00434 *
00435 * @details
00436 * Formats a flags bitmask into a compact pipe-separated string showing all
00437 * active flags. If no flags are set (value == 0), returns "NONE".
00438 *
00439 * @par Flag Mapping
00440 * | Flag Bit | String | Description |
00441 * |-----|-----|-----|
00442 * | k_frame_flag_requires_ack (0x01) | "ACK" | Acknowledgment required |
00443 * | k_frame_flag_retransmit (0x02) | "RETX" | Retransmission |
00444 * | k_frame_flag_priority (0x04) | "PRI" | High priority |
00445 * | k_frame_flag_fec_enabled (0x08) | "FEC" | FEC encoding |
00446 * | k_frame_flag_soft_nack (0x10) | "SOFT" | Soft NACK |
00447 * | k_frame_flag_none (0x00) | "NONE" | No flags set |
00448 *
00449 * @par Output Examples
00450 * | Input Flags | Output String |
00451 * |-----|-----|
00452 * | 0x00 | "NONE" |
00453 * | 0x01 | "ACK" |
00454 * | 0x03 | "ACK|RETX" |
00455 * | 0x07 | "ACK|RETX|PRI" |
00456 * | 0x1F | "ACK|RETX|PRI|FEC|SOFT" |
00457 *
00458 * @param[in] flags Frame flags bitmask
00459 * - Value from rx_frame_flags_t or combination thereof
00460 * - 0 produces "NONE"
00461 * @param[out] buffer Output buffer for formatted string
00462 * - Must be valid non-nullptr
00463 * - Caller maintains ownership
00464 * - Will be null-terminated
00465 * @param[in] buffer_len Size of output buffer in bytes
00466 * - Minimum recommended: 32 bytes
00467 * - Must be > 0

```

```

00468 *
00469 * @return Pointer to buffer with formatted flags string
00470 * @retval buffer On success (contains formatted string)
00471 * @retval "" (empty string) If buffer is nullptr or buffer_len is 0
00472 *
00473 * @pre buffer != nullptr (or returns empty string)
00474 * @pre buffer_len > 0 (or returns empty string)
00475 *
00476 * @post buffer is null-terminated (if valid)
00477 * @post Return value == buffer (if valid)
00478 *
00479 * @note Thread-safe: output written to caller-provided buffer
00480 * @note Reentrant: no static mutable state
00481 *
00482 * @warning Buffer must be large enough for all flags + separators
00483 * @warning Returns empty string literal "" if buffer is nullptr
00484 *
00485 * @par Basic Usage Example
00486 * @code
00487 * char flags_buf[32];
00488 * uint8_t flags = k_frame_flag_requires_ack | k_frame_flag_priority;
00489 *
00490 * const char* str = rx_frame_ascii_flags_str(flags, flags_buf, sizeof(flags_buf));
00491 * // str == "ACK|PRI"
00492 * // flags_buf contains "ACK|PRI\0"
00493 * @endcode
00494 *
00495 * @par Error Handling Example
00496 * @code
00497 * // NULL buffer returns empty string
00498 * const char* empty = rx_frame_ascii_flags_str(0x01, nullptr, 0);
00499 * // empty == ""
00500 *
00501 * // Zero-length buffer returns empty string
00502 * char tiny[1];
00503 * const char* also_empty = rx_frame_ascii_flags_str(0x01, tiny, 0);
00504 * // also_empty == ""
00505 * @endcode
00506 *
00507 * @par Performance
00508 * - Execution time: O(n) where n = number of flag bits checked
00509 * - Stack usage: ~10 bytes
00510 *
00511 * @see rx_frame_flags_t Flag definitions
00512 * @see rx_frame_ascii_format() Uses this function internally
00513 *
00514 * @since Version 1.0.0
00515 */
00516 const char* rx_frame_ascii_flags_str(uint8_t flags, char* buffer, uint32_t buffer_len);
00517
00518 #ifdef __cplusplus
00519 }
00520 #endif

```

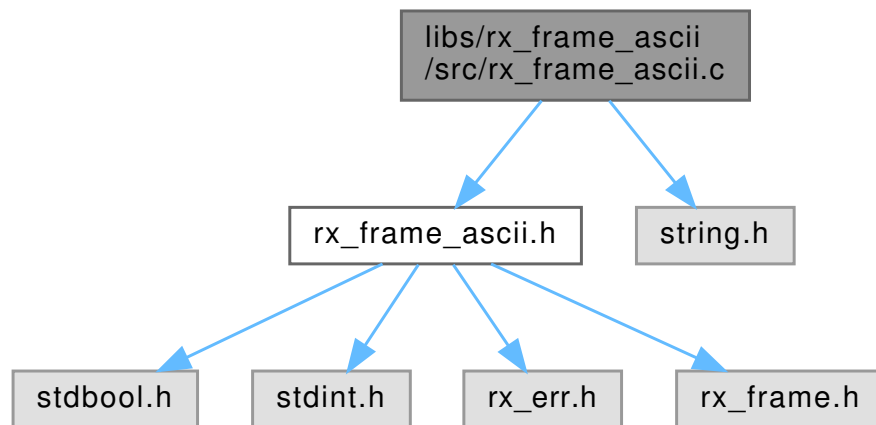
6.3 libs/rx_frame_ascii/src/rx_frame_ascii.c File Reference

Frame ASCII Formatter Implementation.

```
#include "rx_frame_ascii.h"
```

```
#include <string.h>
```

Include dependency graph for rx_frame_ascii.c:



Enumerations

- enum `format_constants_t`: `uint16_t` {
`k_hex_digit_shift` = 4, `k_hex_digit_mask` = 0xF, `k_printable_min` = 0x20, `k_printable_max` = 0x7E,
`k_bytes_per_line` = 16, `k_min_output_size` = 64, `k_hex_space_gap` = 2, `k_mask_u8` = 0xFF,
`k_mask_u16` = 0xFFFF, `k_decimal_base` = 10, `k_u16_decimal_max_digits` = 5, `k_flags_buf_size` = 32 }
String formatting constants for hex and decimal conversion.
- enum `buffer_size_constants_t`: `uint8_t` {
`k_empty_buffer` = 0, `k_null_terminator` = 1, `k_hex_chars_per_byte` = 2, `k_hex_chars_per_u16` = 4,
`k_hex_chars_per_u32` = 8, `k_u32_high_u16_shift` = 16 }
Buffer size and offset constants for string building.
- enum `decimal_buf_constants_t`: `uint8_t` { `k_u16_decimal_buf_size` = `k_u16_decimal_max_digits` + 1 }
Decimal conversion buffer size constants.

Functions

- static `uint32_t` `internal_append_str` (`char` *buf, `uint32_t` pos, `uint32_t` max, const `char` *str)
Append string to buffer with bounds checking.
- static `uint32_t` `internal_append_char` (`char` *buf, `uint32_t` pos, `uint32_t` max, `char` c)
Append single character to buffer with bounds checking.
- static `uint32_t` `internal_append_hex_byte` (`char` *buf, `uint32_t` pos, `uint32_t` max, `uint8_t` byte)
Append byte as two hex characters to buffer.
- static `uint32_t` `internal_append_hex_u16` (`char` *buf, `uint32_t` pos, `uint32_t` max, `uint16_t` value)
Append uint16_t as four hex characters to buffer.
- static `uint32_t` `internal_append_hex_u32` (`char` *buf, `uint32_t` pos, `uint32_t` max, `uint32_t` value)
Append uint32_t as eight hex characters to buffer.
- static `uint32_t` `internal_append_decimal_u16` (`char` *buf, `uint32_t` pos, `uint32_t` max, `uint16_t` value)
Append uint16_t as decimal digits to buffer.
- static `bool` `internal_is_printable` (`uint8_t` c)
Check if byte value is printable ASCII character.
- static `uint32_t` `internal_format_payload` (`char` *buf, `uint32_t` pos, `uint32_t` max, const `uint8_t` *payload, `uint16_t` length)
Format payload as hex dump with ASCII sidebar.
- static `uint32_t` `internal_format_header` (`char` *buf, `uint32_t` pos, `uint32_t` max, const `rx_frame_header_t` *header, `bool` is_tx)
Format frame header metadata as single text line.
- const `char` * `rx_frame_ascii_type_name` (`rx_frame_type_t` type)

- Convert frame type enum to human-readable string.*

static uint32_t [internal_append_flag](#) (char *buffer, uint32_t pos, uint32_t max, bool *first, uint8_t flags, uint8_t flag_bit, const char *flag_name)

Append a single flag name with pipe separator if needed.
- const char * [rx_frame_ascii_flags_str](#) (uint8_t flags, char *buffer, uint32_t buffer_len)

Convert frame flags bitmask to human-readable string.
- rx_err_t [rx_frame_ascii_format](#) (const rx_frame_t *frame, bool is_tx, char *output, uint32_t output_max_len, uint32_t *output_len)

Format frame as human-readable ASCII string.

Variables

- static const char [s_hex_chars](#) [] = "0123456789ABCDEF"

Lookup table for hexadecimal digit characters.

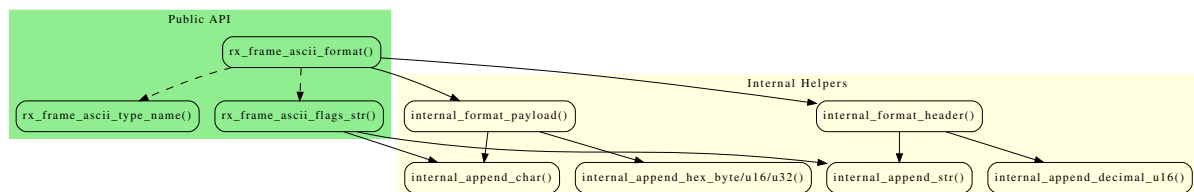
6.3.1 Detailed Description

Frame ASCII Formatter Implementation.

Implements the ASCII frame formatting functions declared in [rx_frame_ascii.h](#). This module converts binary protocol frames into human-readable text for debugging and diagnostic output.

Implementation Overview

The implementation uses a functional decomposition approach with small, focused helper functions for each formatting task:



Design Decisions

- No `snprintf/printf`:** Avoids libc dependencies, non-reentrant functions, and format string vulnerabilities. Custom formatters are safer and smaller.
- Position-based appending:** All append helpers track position and bounds, preventing buffer overflows automatically.
- Static lookup table:** Hex digits use compile-time lookup for speed.
- Consistent signature:** All append helpers follow (buf, pos, max, value) convention for predictable composition.

Memory Safety

Every function validates bounds before writing. Output buffer overflow is impossible if functions are used correctly. Truncation is detected and reported via return value.

Performance Optimizations

- Hex digit lookup table (`s_hex_chars`) avoids arithmetic
- Direct character output avoids format string parsing
- Single-pass formatting (no temporary buffers except small decimal conversion)
- All loops have static upper bounds (NASA Rule 2)

NASA Power of 10 Compliance

- Rule 1: [OK] No goto, setjmp, or recursion
- Rule 2: [OK] All loops bounded (payload length, constant limits)
- Rule 3: [OK] No dynamic memory allocation
- Rule 4: [OK] Functions under 60 lines
- Rule 5: [OK] Input validation on all functions
- Rule 6: [OK] Variables at smallest scope
- Rule 7: [OK] Return values checked
- Rule 8: [OK] C23 typed enums for constants
- Rule 10: [OK] Compiles with `-Wall -Wextra -Werror`

See also

[rx_frame_ascii.h](#) Public API documentation

Author

STAR Team

Date

2026-01-26

Copyright

Copyright (c) 2026 STAR Project. MIT License.

Since

Version 1.0.0

Definition in file [rx_frame_ascii.c](#).

6.3.2 Enumeration Type Documentation

6.3.2.1 `buffer_size_constants_t`

enum `buffer_size_constants_t` : `uint8_t`

Buffer size and offset constants for string building.

Constants for buffer boundary calculations and hex character sizing. All values fit in `uint8_t` for efficient comparison operations.

Since

Version 1.0.0

Enumerator

<code>k_empty_buffer</code>	Sentinel for empty/invalid buffer size. Value: 0
-----------------------------	---

k_null_terminator	Space required for null terminator byte. Value: 1 byte
k_hex_chars_per_byte	Hex characters per byte (e.g., "FF"). Value: 2
k_hex_chars_per_u16	Hex characters for uint16_t (e.g., "FFFF"). Value: 4
k_hex_chars_per_u32	Hex characters for uint32_t (e.g., "FFFFFFFF"). Value: 8
k_u32_high_u16_shift	Bit shift to extract high uint16 from uint32. Value: 16 bits

Definition at line 176 of file [rx_frame_ascii.c](#).

```

00176     : uint8_t {
00177 k_empty_buffer = 0, /**< Sentinel for empty/invalid buffer size.
00178     @par Value: 0 */
00179
00180 k_null_terminator = 1, /**< Space required for null terminator byte.
00181     @par Value: 1 byte */
00182
00183 k_hex_chars_per_byte = 2, /**< Hex characters per byte (e.g., "FF").
00184     @par Value: 2 */
00185
00186 k_hex_chars_per_u16 = 4, /**< Hex characters for uint16_t (e.g., "FFFF").
00187     @par Value: 4 */
00188
00189 k_hex_chars_per_u32 = 8, /**< Hex characters for uint32_t (e.g., "FFFFFFFF").
00190     @par Value: 8 */
00191
00192 k_u32_high_u16_shift = 16, /**< Bit shift to extract high uint16 from uint32.
00193     @par Value: 16 bits */
00194 } buffer_size_constants_t;

```

6.3.2.2 decimal_buf_constants_t

enum [decimal_buf_constants_t](#) : uint8_t

Decimal conversion buffer size constants.

Defines buffer sizes for temporary decimal string conversion.

Since

Version 1.0.0

Enumerator

k_u16_decimal_buf_size	Buffer size for uint16_t decimal with null terminator (6 bytes). Calculation: 5 digits + 1 null
------------------------	--

Definition at line 205 of file [rx_frame_ascii.c](#).

```
00205      : uint8_t {
00206  k_u16_decimal_buf_size = k_u16_decimal_max_digits + 1, /**< Buffer size for uint16_t decimal
00207      with null terminator (6 bytes).
00208      @par Calculation: 5 digits + 1 null */
00209 } decimal_buf_constants_t;
```

6.3.2.3 format_constants_t

enum [format_constants_t](#) : uint16_t

String formatting constants for hex and decimal conversion.

Internal constants used by the formatting helper functions. These define bit manipulation parameters, ASCII character ranges, and output formatting dimensions.

Hex Digit Extraction

To convert a byte to two hex characters:

- Upper nibble: (byte >> [k_hex_digit_shift](#)) & [k_hex_digit_mask](#)
- Lower nibble: byte & [k_hex_digit_mask](#)

Printable ASCII Range

Characters 0x20 (space) through 0x7E (~) are printable. Outside this range, the hex dump ASCII sidebar displays '.' as a placeholder.

Since

Version 1.0.0

Enumerator

k_hex_digit_shift	Bits to shift for upper hex nibble extraction. Value: 4 (half a byte = 4 bits)
-------------------	---

k_hex_digit_mask	Mask for extracting single hex digit (4 bits). Value: 0x0F (binary: 00001111)
k_printable_min	Minimum printable ASCII character (space). Value: 0x20 (32 decimal) See also internal_is_printable()
k_printable_max	Maximum printable ASCII character (tilde). Value: 0x7E (126 decimal) See also internal_is_printable()
k_bytes_per_line	Bytes per line in hex dump output. Matches standard xxd/hexdump format. Value: 16 bytes
k_min_output_size	Minimum output buffer size accepted. Fits header + CRC with empty payload. Value: 64 bytes
k_hex_space_gap	Spaces between hex and ASCII sections. Value: 2 spaces
k_mask_u8	Mask for uint8_t extraction from larger type. Value: 0xFF (8 bits)
k_mask_u16	Mask for uint16_t extraction from uint32_t. Value: 0xFFFF (16 bits)
k_decimal_base	Decimal number base for digit extraction. Value: 10

k_u16_decimal_max_digits	Maximum decimal digits for uint16_t (65535). Value: 5
k_flags_buf_size	Buffer size for flags string formatting. Fits "ACK RETX PRI FEC SOFT" + null. Value: 32 bytes

Definition at line 123 of file `rx_frame_ascii.c`.

```

00123     : uint16_t {
00124 k_hex_digit_shift = 4, /**< Bits to shift for upper hex nibble extraction.
00125                        @par Value: 4 (half a byte = 4 bits) */
00126
00127 k_hex_digit_mask = 0xF, /**< Mask for extracting single hex digit (4 bits).
00128                        @par Value: 0x0F (binary: 00001111) */
00129
00130 k_printable_min = 0x20, /**< Minimum printable ASCII character (space).
00131                        @par Value: 0x20 (32 decimal)
00132                        @see internal_is_printable() */
00133
00134 k_printable_max = 0x7E, /**< Maximum printable ASCII character (tilde).
00135                        @par Value: 0x7E (126 decimal)
00136                        @see internal_is_printable() */
00137
00138 k_bytes_per_line = 16, /**< Bytes per line in hex dump output.
00139                        Matches standard xxd/hexdump format.
00140                        @par Value: 16 bytes */
00141
00142 k_min_output_size = 64, /**< Minimum output buffer size accepted.
00143                        Fits header + CRC with empty payload.
00144                        @par Value: 64 bytes */
00145
00146 k_hex_space_gap = 2, /**< Spaces between hex and ASCII sections.
00147                        @par Value: 2 spaces */
00148
00149 k_mask_u8 = 0xFF, /**< Mask for uint8_t extraction from larger type.
00150                        @par Value: 0xFF (8 bits) */
00151
00152 k_mask_u16 = 0xFFFF, /**< Mask for uint16_t extraction from uint32_t.
00153                        @par Value: 0xFFFF (16 bits) */
00154
00155 k_decimal_base = 10, /**< Decimal number base for digit extraction.
00156                        @par Value: 10 */
00157
00158 k_u16_decimal_max_digits = 5, /**< Maximum decimal digits for uint16_t (65535).
00159                        @par Value: 5 */
00160
00161 k_flags_buf_size = 32, /**< Buffer size for flags string formatting.
00162                        Fits "ACK|RETX|PRI|FEC|SOFT" + null.
00163                        @par Value: 32 bytes */
00164 } format_constants_t;

```

6.3.3 Function Documentation

6.3.3.1 rx_frame_ascii_flags_str()

```

const char * rx_frame_ascii_flags_str (
    uint8_t flags,
    char * buffer,
    uint32_t buffer_len)

```

Convert frame flags bitmask to human-readable string.

Get frame flags as human-readable string.

Implementation of [rx_frame_ascii_flags_str\(\)](#). Iterates through known flag bits, appending names with pipe separators.

Parameters

in	<i>flags</i>	Frame flags bitmask
----	--------------	---------------------

out	<i>buffer</i>	Output buffer for formatted string
in	<i>buffer_len</i>	Size of output buffer in bytes

Returns

Pointer to buffer (or "" if buffer is nullptr)

Note

Flags formatted as pipe-separated: "ACK|RETX|PRI"

See also

[rx_frame_ascii.h](#) for full documentation

Definition at line 874 of file [rx_frame_ascii.c](#).

```

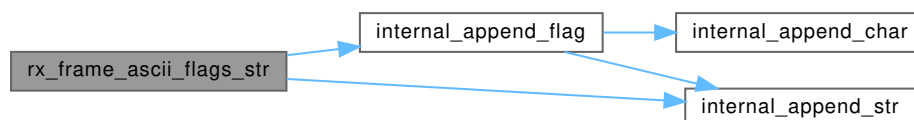
00875 {
00876     if (buffer == nullptr || buffer_len == k_empty_buffer) {
00877         return "";
00878     }
00879     uint32_t pos = 0;
00880     if (flags == k_frame_flag_none) {
00881         pos = internal_append_str(buffer, pos, buffer_len, "NONE");
00882         buffer[pos] = '\0';
00883         return buffer;
00884     }
00885     bool first = true;
00886     /* Use helper to append each flag */
00887     pos =
00888         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_requires_ack, "ACK");
00889     pos =
00890         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_retransmit, "RETX");
00891     pos = internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_priority, "PRI");
00892     pos =
00893         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_fec_enabled, "FEC");
00894     pos =
00895         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_soft_nack, "SOFT");
00896     buffer[pos] = '\0';
00897     return buffer;
00898 }

```

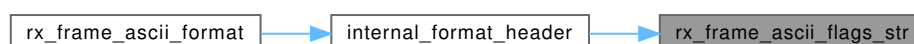
References [internal_append_flag\(\)](#), [internal_append_str\(\)](#), and [k_empty_buffer](#).

Referenced by [internal_format_header\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.3.2 rx_frame_ascii_format()

```
rx_err_t rx_frame_ascii_format (
    const rx_frame_t * frame,
    bool is_tx,
    char * output,
    uint32_t output_max_len,
    uint32_t * output_len) [nodiscard]
```

Format frame as human-readable ASCII string.

Format binary frame as human-readable ASCII.

Implementation of [rx_frame_ascii_format\(\)](#). Orchestrates header, payload, and CRC formatting using internal helper functions.

Implementation Notes

- Validates all inputs before any output
- Calls [internal_format_header\(\)](#) for metadata line
- Calls [internal_format_payload\(\)](#) for hex dump
- Appends CRC footer directly
- Detects truncation and returns k_rx_err_no_mem

Parameters

in	<i>frame</i>	Frame to format (must not be NULL)
in	<i>is_tx</i>	Direction (true=TX, false=RX)
out	<i>output</i>	Output buffer for formatted string
in	<i>output_max_len</i>	Maximum output buffer size
out	<i>output_len</i>	Pointer to receive actual output length

Returns

k_rx_ok on success
k_rx_err_invalid_arg if frame/output/output_len is nullptr
k_rx_err_no_mem if output buffer is too small

See also

[rx_frame_ascii.h](#) for full documentation

Definition at line 931 of file [rx_frame_ascii.c](#).

```
00936 {
00937     /* Rule 5: Pre-condition validation */
00938     if (frame == nullptr) {
00939         return k_rx_err_invalid_arg;
00940     }
00941     if (output == nullptr) {
00942         return k_rx_err_invalid_arg;
00943     }
00944     if (output_len == nullptr) {
00945         return k_rx_err_invalid_arg;
00946     }
```

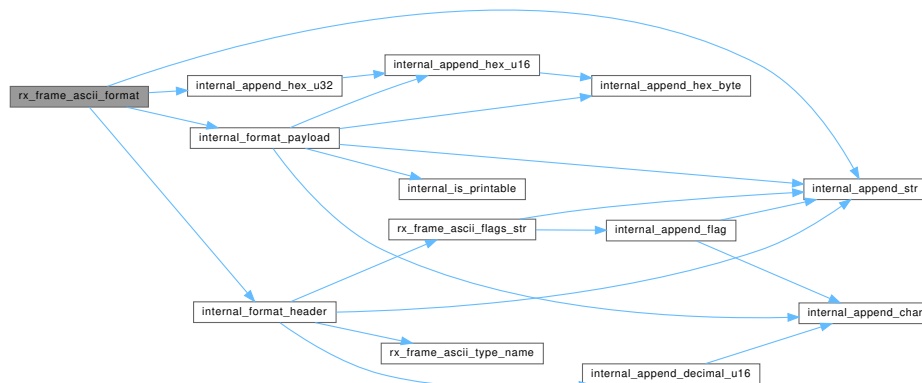
```

00947 if (output_max_len < k_min_output_size) {
00948     return k_rx_err_no_mem;
00949 }
00950
00951 /* Validate payload length does not exceed frame capacity */
00952 if (frame->header.length > k_frame_max_payload) {
00953     return k_rx_err_invalid_arg;
00954 }
00955
00956 uint32_t pos = 0;
00957
00958 /* Format header metadata */
00959 pos = internal_format_header(output, pos, output_max_len, &frame->header, is_tx);
00960
00961 /* Payload hex dump - use validated length to prevent out-of-bounds reads */
00962 pos = internal_format_payload(output, pos, output_max_len, frame->payload, frame->header.length);
00963
00964 /* CRC */
00965 pos = internal_append_str(output, pos, output_max_len, "[CRC] ");
00966 pos = internal_append_hex_u32(output, pos, output_max_len, frame->crc);
00967 pos = internal_append_str(output, pos, output_max_len, "\r\n");
00968
00969 /* Rule 5: Post-condition validation - check for truncation */
00970 /* If pos >= output_max_len, we don't have room for null terminator */
00971 if (pos >= output_max_len) {
00972     /* Buffer was too small - data was truncated */
00973     if (output_max_len > 0) {
00974         output[output_max_len - 1] = '\0'; /* Null terminate at buffer end */
00975     }
00976     return k_rx_err_no_mem;
00977 }
00978
00979 /* Null terminate */
00980 output[pos] = '\0';
00981
00982 *output_len = pos;
00983
00984 return k_rx_ok;
00985 }

```

References [internal_append_hex_u32\(\)](#), [internal_append_str\(\)](#), [internal_format_header\(\)](#), [internal_format_payload\(\)](#), and [k_min_output_size](#).

Here is the call graph for this function:



6.3.3.3 rx_frame_ascii_type_name()

```

const char * rx_frame_ascii_type_name (
    rx_frame_type_t type)

```

Convert frame type enum to human-readable string.

Get frame type as human-readable string.

Implementation of [rx_frame_ascii_type_name\(\)](#). Uses switch statement for O(1) lookup with compile-time string literals.

Parameters

in	<i>type</i>	Frame type enum value
----	-------------	-----------------------

Returns

String representation ("PING", "PONG", "COMMAND", "RESPONSE", "ACK", "NACK", "RESET_ACK", "RESET", or "UNKNOWN")

See also

[rx_frame_ascii.h](#) for full documentation

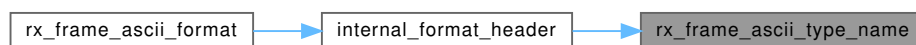
Definition at line 776 of file [rx_frame_ascii.c](#).

```

00777 {
00778     switch (type) {
00779         case k_frame_type_ping:
00780             return "PING";
00781         case k_frame_type_pong:
00782             return "PONG";
00783         case k_frame_type_command:
00784             return "COMMAND";
00785         case k_frame_type_response:
00786             return "RESPONSE";
00787         case k_frame_type_ack:
00788             return "ACK";
00789         case k_frame_type_nack:
00790             return "NACK";
00791         case k_frame_type_reset_ack:
00792             return "RESET_ACK";
00793         case k_frame_type_reset:
00794             return "RESET";
00795         default:
00796             return "UNKNOWN";
00797     }
00798 }
```

Referenced by [internal_format_header\(\)](#).

Here is the caller graph for this function:



6.3.4 Variable Documentation

6.3.4.1 s_hex_chars

```
const char s_hex_chars[] = "0123456789ABCDEF" [static]
```

Lookup table for hexadecimal digit characters.

Static constant lookup table mapping nibble values (0-15) to their ASCII hexadecimal character representation ('0'-'9', 'A'-'F'). Uses uppercase hex letters for consistency with standard debugger output.

Usage

```
uint8_t nibble = (byte » 4) & 0x0F; // Extract upper nibble
char hex_char = s_hex_chars[nibble]; // Convert to hex character
```

Memory

17 bytes (16 characters + null terminator) in .rodata section

Note

Thread-safe: read-only constant data

Since

Version 1.0.0

Definition at line 233 of file `rx_frame_ascii.c`.

Referenced by `internal_append_hex_byte()`.

6.4 rx_frame_ascii.c

[Go to the documentation of this file.](#)

```
00001 /* lib/rx_frame_ascii/src/rx_frame_ascii.c */
00002
00003 /**
00004  * @file rx_frame_ascii.c
00005  * @brief Frame ASCII Formatter Implementation
00006  *
00007  * @details
00008  * Implements the ASCII frame formatting functions declared in rx_frame_ascii.h.
00009  * This module converts binary protocol frames into human-readable text for
00010  * debugging and diagnostic output.
00011  *
00012  * @par Implementation Overview
00013  * The implementation uses a functional decomposition approach with small,
00014  * focused helper functions for each formatting task:
00015  *
00016  * @dot
00017  * digraph impl_structure {
00018  *   rankdir=TB;
00019  *   node [shape=box, style=rounded];
00020  *
00021  *   subgraph cluster_public {
00022  *     label="Public API";
00023  *     style=filled;
00024  *     color=lightgreen;
00025  *     format [label="rx_frame_ascii_format()"];
00026  *     type_name [label="rx_frame_ascii_type_name()"];
00027  *     flags_str [label="rx_frame_ascii_flags_str()"];
00028  *   }
00029  *
00030  *   subgraph cluster_helpers {
00031  *     label="Internal Helpers";
00032  *     style=filled;
00033  *     color=lightyellow;
00034  *     append_str [label="internal_append_str()"];
00035  *     append_char [label="internal_append_char()"];
00036  *     append_hex [label="internal_append_hex_byte/u16/u32()"];
00037  *     append_dec [label="internal_append_decimal_u16()"];
00038  *     format_header [label="internal_format_header()"];
00039  *     format_payload [label="internal_format_payload()"];
00040  *   }
00041  *
00042  *   format -> format_header;
00043  *   format -> format_payload;
00044  *   format -> type_name [style=dashed];
```

```

00045 * format -> flags_str [style=dashed];
00046 * format_header -> append_str;
00047 * format_header -> append_dec;
00048 * format_payload -> append_hex;
00049 * format_payload -> append_char;
00050 * flags_str -> append_str;
00051 * flags_str -> append_char;
00052 * }
00053 * @enddot
00054 *
00055 * @par Design Decisions
00056 * - **No snprintf/printf**: Avoids libc dependencies, non-reentrant functions,
00057 *   and format string vulnerabilities. Custom formatters are safer and smaller.
00058 * - **Position-based appending**: All append helpers track position and bounds,
00059 *   preventing buffer overflows automatically.
00060 * - **Static lookup table**: Hex digits use compile-time lookup for speed.
00061 * - **Consistent signature**: All append helpers follow (buf, pos, max, value)
00062 *   convention for predictable composition.
00063 *
00064 * @par Memory Safety
00065 * Every function validates bounds before writing. Output buffer overflow is
00066 * impossible if functions are used correctly. Truncation is detected and
00067 * reported via return value.
00068 *
00069 * @par Performance Optimizations
00070 * - Hex digit lookup table (s_hex_chars) avoids arithmetic
00071 * - Direct character output avoids format string parsing
00072 * - Single-pass formatting (no temporary buffers except small decimal conversion)
00073 * - All loops have static upper bounds (NASA Rule 2)
00074 *
00075 * @par NASA Power of 10 Compliance
00076 * - Rule 1: [OK] No goto, setjmp, or recursion
00077 * - Rule 2: [OK] All loops bounded (payload length, constant limits)
00078 * - Rule 3: [OK] No dynamic memory allocation
00079 * - Rule 4: [OK] Functions under 60 lines
00080 * - Rule 5: [OK] Input validation on all functions
00081 * - Rule 6: [OK] Variables at smallest scope
00082 * - Rule 7: [OK] Return values checked
00083 * - Rule 8: [OK] C23 typed enums for constants
00084 * - Rule 10: [OK] Compiles with -Wall -Wextra -Werror
00085 *
00086 * @see rx_frame_ascii.h Public API documentation
00087 *
00088 * @author STAR Team
00089 * @date 2026-01-26
00090 * @copyright Copyright (c) 2026 STAR Project. MIT License.
00091 * @since Version 1.0.0
00092 */
00093
00094 #include "rx_frame_ascii.h"
00095
00096 #include <string.h>
00097
00098 /*
=====
00099 * Private Constants
00100 *
=====
00101 */
00102
00103 /**
00104 * @enum format_constants_t
00105 * @brief String formatting constants for hex and decimal conversion
00106 *
00107 * @details
00108 * Internal constants used by the formatting helper functions. These define
00109 * bit manipulation parameters, ASCII character ranges, and output formatting
00110 * dimensions.
00111 *
00112 * @par Hex Digit Extraction
00113 * To convert a byte to two hex characters:
00114 * - Upper nibble: `(byte » k_hex_digit_shift) & k_hex_digit_mask`
00115 * - Lower nibble: `byte & k_hex_digit_mask`
00116 *
00117 * @par Printable ASCII Range
00118 * Characters 0x20 (space) through 0x7E (~) are printable. Outside this range,
00119 * the hex dump ASCII sidebar displays `.` as a placeholder.
00120 *
00121 * @since Version 1.0.0
00122 */
00123 typedef enum : uint16_t {
00124     k_hex_digit_shift = 4, /**< Bits to shift for upper hex nibble extraction.
00125                             @par Value: 4 (half a byte = 4 bits) */
00126
00127     k_hex_digit_mask = 0xF, /**< Mask for extracting single hex digit (4 bits).
00128                             @par Value: 0x0F (binary: 00001111) */
00129 }

```

```

00130 k_printable_min = 0x20, /**< Minimum printable ASCII character (space).
00131 @par Value: 0x20 (32 decimal)
00132 @see internal_is_printable() */
00133
00134 k_printable_max = 0x7E, /**< Maximum printable ASCII character (tilde).
00135 @par Value: 0x7E (126 decimal)
00136 @see internal_is_printable() */
00137
00138 k_bytes_per_line = 16, /**< Bytes per line in hex dump output.
00139 Matches standard xxd/hexdump format.
00140 @par Value: 16 bytes */
00141
00142 k_min_output_size = 64, /**< Minimum output buffer size accepted.
00143 Fits header + CRC with empty payload.
00144 @par Value: 64 bytes */
00145
00146 k_hex_space_gap = 2, /**< Spaces between hex and ASCII sections.
00147 @par Value: 2 spaces */
00148
00149 k_mask_u8 = 0xFF, /**< Mask for uint8_t extraction from larger type.
00150 @par Value: 0xFF (8 bits) */
00151
00152 k_mask_u16 = 0xFFFF, /**< Mask for uint16_t extraction from uint32_t.
00153 @par Value: 0xFFFF (16 bits) */
00154
00155 k_decimal_base = 10, /**< Decimal number base for digit extraction.
00156 @par Value: 10 */
00157
00158 k_u16_decimal_max_digits = 5, /**< Maximum decimal digits for uint16_t (65535).
00159 @par Value: 5 */
00160
00161 k_flags_buf_size = 32, /**< Buffer size for flags string formatting.
00162 Fits "ACK|RETX|PRI|FEC|SOFT" + null.
00163 @par Value: 32 bytes */
00164 } format_constants_t;
00165
00166 /**
00167 * @enum buffer_size_constants_t
00168 * @brief Buffer size and offset constants for string building
00169 *
00170 * @details
00171 * Constants for buffer boundary calculations and hex character sizing.
00172 * All values fit in uint8_t for efficient comparison operations.
00173 *
00174 * @since Version 1.0.0
00175 */
00176 typedef enum : uint8_t {
00177     k_empty_buffer = 0, /**< Sentinel for empty/invalid buffer size.
00178 @par Value: 0 */
00179
00180     k_null_terminator = 1, /**< Space required for null terminator byte.
00181 @par Value: 1 byte */
00182
00183     k_hex_chars_per_byte = 2, /**< Hex characters per byte (e.g., "FF").
00184 @par Value: 2 */
00185
00186     k_hex_chars_per_u16 = 4, /**< Hex characters for uint16_t (e.g., "FFFF").
00187 @par Value: 4 */
00188
00189     k_hex_chars_per_u32 = 8, /**< Hex characters for uint32_t (e.g., "FFFFFFFF").
00190 @par Value: 8 */
00191
00192     k_u32_high_u16_shift = 16, /**< Bit shift to extract high uint16 from uint32.
00193 @par Value: 16 bits */
00194 } buffer_size_constants_t;
00195
00196 /**
00197 * @enum decimal_buf_constants_t
00198 * @brief Decimal conversion buffer size constants
00199 *
00200 * @details
00201 * Defines buffer sizes for temporary decimal string conversion.
00202 *
00203 * @since Version 1.0.0
00204 */
00205 typedef enum : uint8_t {
00206     k_u16_decimal_buf_size = k_u16_decimal_max_digits + 1, /**< Buffer size for uint16_t decimal
00207 with null terminator (6 bytes).
00208 @par Calculation: 5 digits + 1 null */
00209 } decimal_buf_constants_t;
00210
00211 /**
00212 * @var s_hex_chars
00213 * @brief Lookup table for hexadecimal digit characters
00214 *
00215 * @details
00216 * Static constant lookup table mapping nibble values (0-15) to their ASCII

```



```

00217 * hexadecimal character representation ('0'-'9', 'A'-'F'). Uses uppercase
00218 * hex letters for consistency with standard debugger output.
00219 *
00220 * @par Usage
00221 * @code
00222 * uint8_t nibble = (byte » 4) & 0x0F; // Extract upper nibble
00223 * char hex_char = s_hex_chars[nibble]; // Convert to hex character
00224 * @endcode
00225 *
00226 * @par Memory
00227 * 17 bytes (16 characters + null terminator) in .rodata section
00228 *
00229 * @note Thread-safe: read-only constant data
00230 *
00231 * @since Version 1.0.0
00232 */
00233 static const char s_hex_chars[] = "0123456789ABCDEF";
00234
00235 /*
=====
00236 * Private Helper Functions
00237 *
=====
00238 */
00239
00240 /**
00241 * @defgroup frame_ascii_helpers Internal Helper Functions
00242 * @{
00243 *
00244 * @brief String building helper functions with bounds checking
00245 *
00246 * @details
00247 * All append helpers follow a consistent signature pattern for composability:
00248 * ```c
00249 * uint32_t internal_append_xxx(char* buf, uint32_t pos, uint32_t max, <value>);
00250 * ```
00251 *
00252 * @par Parameters
00253 * - **buf**: Output buffer pointer (may be NULL for safety)
00254 * - **pos**: Current write position in buffer
00255 * - **max**: Maximum buffer size (total capacity)
00256 * - **value**: Data to append (varies by function)
00257 *
00258 * @par Return Value
00259 * All helpers return the updated position after appending. If the append
00260 * would overflow, the original position is returned unchanged.
00261 *
00262 * @par Bounds Safety
00263 * - All functions check `pos < max - k_null_terminator` before writing
00264 * - NULL buffer pointer causes immediate return with unchanged position
00265 * - Zero-size buffer causes immediate return with unchanged position
00266 * - No writes beyond `max - 1` to preserve space for null terminator
00267 *
00268 * @par Usage Pattern
00269 * ```c
00270 * uint32_t pos = 0;
00271 * pos = internal_append_str(buf, pos, max, "Hello ");
00272 * pos = internal_append_decimal_u16(buf, pos, max, 42);
00273 * pos = internal_append_char(buf, pos, max, '!');
00274 * buf[pos] = '\0'; // Null terminate
00275 * // Result: "Hello 42!"
00276 * ```
00277 * @}
00278 */
00279
00280 /**
00281 * @brief Append string to buffer with bounds checking
00282 *
00283 * @details
00284 * Copies characters from source string to output buffer until null terminator
00285 * is reached or buffer is full (leaving room for final null terminator).
00286 *
00287 * @param[in,out] buf Output buffer (may be NULL)
00288 * @param[in] pos Current write position
00289 * @param[in] max Maximum buffer size
00290 * @param[in] str Source string to append (may be NULL)
00291 *
00292 * @return Updated position after appending
00293 * @retval pos (unchanged) If buf, str is nullptr, or buffer full
00294 *
00295 * @pre pos < max (for any writing to occur)
00296 *
00297 * @post Characters written in range [pos, return_value)
00298 * @post Buffer NOT null-terminated (caller responsibility)
00299 *
00300 * @note Does not write null terminator
00301 * @note Returns original pos if nothing can be written

```

```

00302 *
00303 * @ingroup frame_ascii_helpers
00304 * @since Version 1.0.0
00305 */
00306 static uint32_t internal_append_str(char* buf, uint32_t pos, uint32_t max, const char* str)
00307 {
00308     /* Rule 5: Pre-condition validation */
00309     if (buf == nullptr || str == nullptr || max == k_empty_buffer || pos >= max) {
00310         return pos;
00311     }
00312     while (*str != '\0' && pos < max - k_null_terminator) {
00313         buf[pos++] = *str++;
00314     }
00315     return pos;
00316 }
00317 }
00318
00319 /**
00320 * @brief Append single character to buffer with bounds checking
00321 *
00322 * @details
00323 * Writes a single character to the buffer at the current position if space
00324 * is available (accounting for null terminator).
00325 *
00326 * @param[in,out] buf Output buffer (may be NULL)
00327 * @param[in] pos Current write position
00328 * @param[in] max Maximum buffer size
00329 * @param[in] c Character to append
00330 *
00331 * @return Updated position after appending
00332 * @retval pos (unchanged) If buf is nullptr or buffer full
00333 * @retval pos + 1 If character was written
00334 *
00335 * @pre pos < max - 1 (for writing to occur)
00336 *
00337 * @post buf[pos] contains c (if written)
00338 *
00339 * @ingroup frame_ascii_helpers
00340 * @since Version 1.0.0
00341 */
00342 static uint32_t internal_append_char(char* buf, uint32_t pos, uint32_t max, char c)
00343 {
00344     /* Rule 5: Pre-condition validation */
00345     if (buf == nullptr || max == k_empty_buffer || pos >= max) {
00346         return pos;
00347     }
00348     if (pos < max - k_null_terminator) {
00349         buf[pos++] = c;
00350     }
00351     return pos;
00352 }
00353 }
00354
00355 /**
00356 * @brief Append byte as two hex characters to buffer
00357 *
00358 * @details
00359 * Converts a single byte to its two-character hexadecimal representation
00360 * (e.g., 0xFF -> "FF") and appends it to the buffer. Uses uppercase hex
00361 * letters via s_hex_chars lookup table.
00362 *
00363 * @param[in,out] buf Output buffer (may be NULL)
00364 * @param[in] pos Current write position
00365 * @param[in] max Maximum buffer size
00366 * @param[in] byte Byte value to convert (0x00-0xFF)
00367 *
00368 * @return Updated position after appending (pos + 2 on success)
00369 * @retval pos (unchanged) If buf is nullptr or insufficient space
00370 *
00371 * @pre pos + 2 <= max - 1 (for writing to occur)
00372 *
00373 * @post buf[pos] contains upper nibble hex char
00374 * @post buf[pos+1] contains lower nibble hex char
00375 *
00376 * @par Example Output
00377 * - byte=0x00 -> "00"
00378 * - byte=0xFF -> "FF"
00379 * - byte=0x5A -> "5A"
00380 *
00381 * @ingroup frame_ascii_helpers
00382 * @since Version 1.0.0
00383 */
00384 static uint32_t internal_append_hex_byte(char* buf, uint32_t pos, uint32_t max, uint8_t byte)
00385 {
00386     /* Rule 5: Pre-condition validation */
00387     if (buf == nullptr || max < k_hex_chars_per_byte || pos >= max) {
00388         return pos;

```

```

00389 }
00390
00391 if (pos < max - k_hex_chars_per_byte) {
00392     buf[pos++] = s_hex_chars[(byte » k_hex_digit_shift) & k_hex_digit_mask];
00393     buf[pos++] = s_hex_chars[(byte & k_hex_digit_mask)];
00394 }
00395 return pos;
00396 }
00397
00398 /**
00399  * @brief Append uint16_t as four hex characters to buffer
00400  *
00401  * @details
00402  * Converts a 16-bit value to its four-character hexadecimal representation
00403  * (e.g., 0xABCD -> "ABCD") in big-endian order (most significant byte first).
00404  *
00405  * @param[in,out] buf Output buffer (may be NULL)
00406  * @param[in] pos Current write position
00407  * @param[in] max Maximum buffer size
00408  * @param[in] value 16-bit value to convert
00409  *
00410  * @return Updated position after appending (pos + 4 on success)
00411  * @retval pos (unchanged) If buf is nullptr or insufficient space
00412  *
00413  * @pre pos + 4 <= max - 1 (for writing to occur)
00414  *
00415  * @post Four hex characters written at buf[pos..pos+3]
00416  *
00417  * @par Example Output
00418  * - value=0x0000 -> "0000"
00419  * - value=0xFFFF -> "FFFF"
00420  * - value=0x1234 -> "1234"
00421  *
00422  * @ingroup frame_ascii_helpers
00423  * @since Version 1.0.0
00424  */
00425 static uint32_t internal_append_hex_u16(char* buf, uint32_t pos, uint32_t max, uint16_t value)
00426 {
00427     /* Rule 5: Pre-condition validation */
00428     if (buf == nullptr || max < k_hex_chars_per_u16 || pos >= max) {
00429         return pos;
00430     }
00431
00432     pos = internal_append_hex_byte(buf, pos, max, (uint8_t)(value » 8));
00433     pos = internal_append_hex_byte(buf, pos, max, (uint8_t)(value & k_mask_u8));
00434     return pos;
00435 }
00436
00437 /**
00438  * @brief Append uint32_t as eight hex characters to buffer
00439  *
00440  * @details
00441  * Converts a 32-bit value to its eight-character hexadecimal representation
00442  * (e.g., 0xDEADBEEF -> "DEADBEEF") in big-endian order. Commonly used for
00443  * CRC-32 values in frame output.
00444  *
00445  * @param[in,out] buf Output buffer (may be NULL)
00446  * @param[in] pos Current write position
00447  * @param[in] max Maximum buffer size
00448  * @param[in] value 32-bit value to convert
00449  *
00450  * @return Updated position after appending (pos + 8 on success)
00451  * @retval pos (unchanged) If buf is nullptr or insufficient space
00452  *
00453  * @pre pos + 8 <= max - 1 (for writing to occur)
00454  *
00455  * @post Eight hex characters written at buf[pos..pos+7]
00456  *
00457  * @par Example Output
00458  * - value=0x00000000 -> "00000000"
00459  * - value=0xFFFFFFFF -> "FFFFFFFF"
00460  * - value=0xDEADBEEF -> "DEADBEEF"
00461  *
00462  * @ingroup frame_ascii_helpers
00463  * @since Version 1.0.0
00464  */
00465 static uint32_t internal_append_hex_u32(char* buf, uint32_t pos, uint32_t max, uint32_t value)
00466 {
00467     /* Rule 5: Pre-condition validation */
00468     if (buf == nullptr || max < k_hex_chars_per_u32 || pos >= max) {
00469         return pos;
00470     }
00471
00472     pos = internal_append_hex_u16(buf, pos, max, (uint16_t)(value » k_u32_high_u16_shift));
00473     pos = internal_append_hex_u16(buf, pos, max, (uint16_t)(value & k_mask_u16));
00474     return pos;
00475 }

```

```

00476
00477 /**
00478  * @brief Append uint16_t as decimal digits to buffer
00479  *
00480  * @details
00481  * Converts a 16-bit value to its decimal string representation (1-5 digits).
00482  * Uses a small stack buffer for digit reversal, avoiding heap allocation.
00483  * Zero value outputs single '0' character.
00484  *
00485  * @par Algorithm
00486  * 1. Extract digits via repeated modulo/division (reverse order)
00487  * 2. Store in temporary stack buffer
00488  * 3. Reverse and append to output buffer
00489  *
00490  * @param[in,out] buf Output buffer (may be NULL)
00491  * @param[in] pos Current write position
00492  * @param[in] max Maximum buffer size
00493  * @param[in] value 16-bit value to convert (0-65535)
00494  *
00495  * @return Updated position after appending
00496  * @retval pos (unchanged) If buf is nullptr or insufficient space
00497  *
00498  * @pre Sufficient space for up to 5 digits
00499  *
00500  * @post Decimal digits written without leading zeros (except for value=0)
00501  *
00502  * @par Example Output
00503  * - value=0 -> "0"
00504  * - value=1 -> "1"
00505  * - value=12345 -> "12345"
00506  * - value=65535 -> "65535"
00507  *
00508  * @par Stack Usage
00509  * ~6 bytes (temp buffer for digit reversal)
00510  *
00511  * @ingroup frame_ascii_helpers
00512  * @since Version 1.0.0
00513  */
00514 static uint32_t internal_append_decimal_u16(char* buf, uint32_t pos, uint32_t max, uint16_t value)
00515 {
00516     /* Rule 5: Pre-condition validation */
00517     if (buf == nullptr || max == k_empty_buffer || pos >= max) {
00518         return pos;
00519     }
00520
00521     char temp[k_u16_decimal_buf_size]; /* Max 5 digits + null for uint16_t */
00522     uint8_t len = 0;
00523
00524     if (value == 0) {
00525         return internal_append_char(buf, pos, max, '0');
00526     }
00527
00528     while (value > 0 && len < k_u16_decimal_max_digits) {
00529         const uint8_t digit = (uint8_t)(value % k_decimal_base);
00530         temp[len++] = (char)('0' + digit);
00531         value /= k_decimal_base;
00532     }
00533
00534     /* Reverse digits into output */
00535     while (len > 0) {
00536         pos = internal_append_char(buf, pos, max, temp[--len]);
00537     }
00538
00539     return pos;
00540 }
00541
00542 /**
00543  * @brief Check if byte value is printable ASCII character
00544  *
00545  * @details
00546  * Tests whether a byte value falls within the printable ASCII range
00547  * (0x20-0x7E inclusive). Used to decide whether to display the actual
00548  * character or a '?' placeholder in hex dump ASCII sidebars.
00549  *
00550  * @par Printable Range
00551  * - 0x20 (space) through 0x7E (~) are printable
00552  * - 0x00-0x1F are control characters (not printable)
00553  * - 0x7F (DEL) is not printable
00554  * - 0x80-0xFF are extended ASCII (not printable in this implementation)
00555  *
00556  * @param[in] c Byte value to test
00557  *
00558  * @return true if printable, false otherwise
00559  * @retval true If c in range [0x20, 0x7E]
00560  * @retval false Otherwise
00561  *
00562  * @note Pure function with no side effects

```

```

00563 *
00564 * @ingroup frame_ascii_helpers
00565 * @since Version 1.0.0
00566 */
00567 static bool internal_is_printable(uint8_t c)
00568 {
00569     return (c >= k_printable_min && c <= k_printable_max);
00570 }
00571
00572 /**
00573 * @brief Format payload as hex dump with ASCII sidebar
00574 *
00575 * @details
00576 * Formats binary payload data as a traditional hex dump with 16 bytes per line,
00577 * offset prefix, and ASCII sidebar showing printable characters (or '?' for
00578 * non-printable bytes).
00579 *
00580 * @par Output Format
00581 * Each line follows this structure:
00582 * ``
00583 * [NNNN] XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX .....
00584 * ``
00585 * Where:
00586 * - `[NNNN]` is the 4-digit hex offset from payload start
00587 * - `XX XX ...` is the hex representation of up to 16 bytes
00588 * - ASCII sidebar shows printable chars or '?' for non-printable
00589 *
00590 * @par Empty Payload Handling
00591 * If length is 0, outputs: `[PAYLOAD] (empty)\r\n`
00592 *
00593 * @param[in,out] buf Output buffer
00594 * @param[in] pos Current position in buffer
00595 * @param[in] max Maximum buffer size
00596 * @param[in] payload Payload data to format (may be NULL if length is 0)
00597 * @param[in] length Payload length in bytes
00598 *
00599 * @return Updated position in buffer
00600 * @retval pos (unchanged) If buf is nullptr or max is 0
00601 *
00602 * @pre buf != nullptr (for any output)
00603 * @pre payload != nullptr if length > 0
00604 *
00605 * @post Hex dump lines written with CRLF line endings
00606 *
00607 * @par Loop Bounds (NASA Rule 2)
00608 * - Outer loop: bounded by `length / k_bytes_per_line + 1` iterations
00609 * - Inner loops: bounded by `k_bytes_per_line` (16) iterations
00610 *
00611 * @ingroup frame_ascii_helpers
00612 * @since Version 1.0.0
00613 */
00614 static uint32_t internal_format_payload(char* buf,
00615                                         uint32_t pos,
00616                                         uint32_t max,
00617                                         const uint8_t* payload,
00618                                         uint16_t length)
00619 {
00620     /* Rule 5: Pre-condition validation */
00621     if (buf == nullptr || max == k_empty_buffer || pos >= max) {
00622         return pos;
00623     }
00624
00625     if (length == 0) {
00626         pos = internal_append_str(buf, pos, max, "[PAYLOAD] (empty)\r\n");
00627         return pos;
00628     }
00629
00630     /* Validate payload is non-NULL if length > 0 */
00631     if (payload == nullptr) {
00632         return pos;
00633     }
00634
00635     uint16_t offset = 0;
00636     while (offset < length) {
00637         /* Start new line with offset */
00638         pos = internal_append_str(buf, pos, max, "");
00639         pos = internal_append_hex_u16(buf, pos, max, offset);
00640         pos = internal_append_str(buf, pos, max, " ");
00641
00642         /* Calculate bytes on this line */
00643         uint16_t line_bytes = length - offset;
00644         if (line_bytes > k_bytes_per_line) {
00645             line_bytes = k_bytes_per_line;
00646         }
00647
00648         /* Hex dump */
00649         for (uint16_t i = 0; i < k_bytes_per_line; i++) {

```

```

00650     if (i < line_bytes) {
00651         pos = internal_append_hex_byte(buf, pos, max, payload[offset + i]);
00652         pos = internal_append_char(buf, pos, max, ' ');
00653     } else {
00654         pos = internal_append_str(buf, pos, max, " "); /* Pad short lines */
00655     }
00656 }
00657
00658 /* ASCII sidebar */
00659 pos = internal_append_char(buf, pos, max, ' ');
00660 for (uint16_t i = 0; i < line_bytes; i++) {
00661     uint8_t c = payload[offset + i];
00662     pos = internal_append_char(buf, pos, max, internal_is_printable(c) ? (char)c : '?');
00663 }
00664
00665 pos = internal_append_str(buf, pos, max, "\r\n");
00666 offset += line_bytes;
00667 }
00668
00669 return pos;
00670 }
00671
00672 /**
00673  * @brief Format frame header metadata as single text line
00674  *
00675  * @details
00676  * Formats the frame header fields into a human-readable summary line
00677  * including direction indicator, sequence number, payload length, frame
00678  * type, and flags.
00679  *
00680  * @par Output Format
00681  * ``
00682  * [TX] seq=12345 len=64 type=COMMAND flags=ACK|PRI
00683  * [RX] seq=12346 len=128 type=RESPONSE flags=NONE
00684  * ``
00685  *
00686  * @par Field Details
00687  * | Field | Format | Example |
00688  * |-----|-----|-----|
00689  * | Direction | [TX] or [RX] | [TX] |
00690  * | Sequence | seq=N (decimal) | seq=12345 |
00691  * | Length | len=N (decimal) | len=64 |
00692  * | Type | type=NAME | type=COMMAND |
00693  * | Flags | flags=F1\F2 | flags=ACK\|PRI |
00694  *
00695  * @param[in,out] buf Output buffer
00696  * @param[in] pos Current position in buffer
00697  * @param[in] max Maximum buffer size
00698  * @param[in] header Frame header to format
00699  * @param[in] is_tx Direction indicator (true=TX, false=RX)
00700  *
00701  * @return Updated position in buffer
00702  * @retval pos (unchanged) If buf or header is nullptr, or max is 0
00703  *
00704  * @pre buf != nullptr
00705  * @pre header != nullptr
00706  *
00707  * @post Header line written with CRLF line ending
00708  * @post pos <= max (invariant maintained)
00709  *
00710  * @ingroup frame_ascii_helpers
00711  * @since Version 1.0.0
00712  */
00713 static uint32_t internal_format_header(char* buf,
00714                                       uint32_t pos,
00715                                       uint32_t max,
00716                                       const rx_frame_header_t* header,
00717                                       bool is_tx)
00718 {
00719     /* Rule 5: Pre-condition validation */
00720     if (buf == nullptr || header == nullptr || max == k_empty_buffer || pos >= max) {
00721         return pos;
00722     }
00723
00724     char flags_buf[k_flags_buf_size];
00725
00726     /* Direction */
00727     pos = internal_append_str(buf, pos, max, is_tx ? "[TX]" : "[RX] ");
00728
00729     /* Sequence number */
00730     pos = internal_append_str(buf, pos, max, "seq=");
00731     pos = internal_append_decimal_u16(buf, pos, max, header->sequence);
00732
00733     /* Length */
00734     pos = internal_append_str(buf, pos, max, "len=");
00735     pos = internal_append_decimal_u16(buf, pos, max, header->length);
00736

```

```

00737 /* Type */
00738 pos = internal_append_str(buf, pos, max, " type=");
00739 pos = internal_append_str(buf, pos, max, rx_frame_ascii_type_name((rx_frame_type_t)header->type));
00740
00741 /* Flags */
00742 pos = internal_append_str(buf, pos, max, " flags=");
00743 (void)rx_frame_ascii_flags_str(header->flags, flags_buf, sizeof(flags_buf));
00744 pos = internal_append_str(buf, pos, max, flags_buf);
00745 pos = internal_append_str(buf, pos, max, "\r\n");
00746
00747 /* Rule 5: Post-condition validation - ensure pos never exceeds max.
00748  * Defensive invariant: internal_append_* currently enforce pos <= max, but this
00749  * guard protects against buffer overflows if future refactors change their behavior. */
00750 if (pos > max) {
00751     pos = max;
00752 }
00753
00754 return pos;
00755 }
00756
00757 /*
=====
00758 * Public Functions
00759 *
=====
00760 */
00761
00762 /**
00763  * @brief Convert frame type enum to human-readable string
00764  *
00765  * @details
00766  * Implementation of rx_frame_ascii_type_name(). Uses switch statement
00767  * for O(1) lookup with compile-time string literals.
00768  *
00769  * @param[in] type Frame type enum value
00770  *
00771  * @return String representation ("PING", "PONG", "COMMAND", "RESPONSE", "ACK", "NACK",
00772  *         "RESET_ACK", "RESET", or "UNKNOWN")
00773  *
00774  * @see rx_frame_ascii.h for full documentation
00775  */
00776 const char* rx_frame_ascii_type_name(rx_frame_type_t type)
00777 {
00778     switch (type) {
00779         case k_frame_type_ping:
00780             return "PING";
00781         case k_frame_type_pong:
00782             return "PONG";
00783         case k_frame_type_command:
00784             return "COMMAND";
00785         case k_frame_type_response:
00786             return "RESPONSE";
00787         case k_frame_type_ack:
00788             return "ACK";
00789         case k_frame_type_nack:
00790             return "NACK";
00791         case k_frame_type_reset_ack:
00792             return "RESET_ACK";
00793         case k_frame_type_reset:
00794             return "RESET";
00795         default:
00796             return "UNKNOWN";
00797     }
00798 }
00799
00800 /**
00801  * @brief Append a single flag name with pipe separator if needed
00802  *
00803  * @details
00804  * Conditionally appends a flag name to the output buffer if the corresponding
00805  * flag bit is set. Manages separator insertion: no separator before first flag,
00806  * pipe separator before subsequent flags.
00807  *
00808  * @par Example Sequence
00809  * 1. flags=ACK|PRI, first call (first=true): appends "ACK", sets first=false
00810  * 2. Same flags, second call: appends "|PRI"
00811  *
00812  * @param[in,out] buffer Output buffer
00813  * @param[in] pos Current position in buffer
00814  * @param[in] max Maximum buffer size
00815  * @param[in,out] first Pointer to first-flag indicator
00816  * - true: No separator needed (this is first flag)
00817  * - false: Prepend '|' separator
00818  * - Set to false after appending a flag
00819  * @param[in] flags Complete flag bitmask to test
00820  * @param[in] flag_bit Specific flag bit to check (e.g., 0x01 for ACK)
00821  * @param[in] flag_name String name of the flag (e.g., "ACK")

```

```

00822 *
00823 * @return Updated position in buffer
00824 * @retval pos (unchanged) If flag_bit not set or first is nullptr
00825 *
00826 * @pre first != nullptr
00827 *
00828 * @post *first set to false if flag was appended
00829 * @post Separator '|' prepended if not first flag
00830 *
00831 * @ingroup frame_ascii_helpers
00832 * @since Version 1.0.0
00833 */
00834 static uint32_t internal_append_flag(char*      buffer,
00835                                     uint32_t   pos,
00836                                     uint32_t   max,
00837                                     bool*      first,
00838                                     uint8_t    flags,
00839                                     uint8_t    flag_bit,
00840                                     const char* flag_name)
00841 {
00842     /* Rule 5: Pre-condition validation */
00843     if (first == nullptr) {
00844         return pos;
00845     }
00846
00847     if (flags & flag_bit) {
00848         if (!*first) {
00849             pos = internal_append_char(buffer, pos, max, '|');
00850         }
00851         pos = internal_append_str(buffer, pos, max, flag_name);
00852         *first = false;
00853     }
00854     return pos;
00855 }
00856
00857 /**
00858 * @brief Convert frame flags bitmask to human-readable string
00859 *
00860 * @details
00861 * Implementation of rx_frame_ascii_flags_str(). Iterates through known
00862 * flag bits, appending names with pipe separators.
00863 *
00864 * @param[in] flags Frame flags bitmask
00865 * @param[out] buffer Output buffer for formatted string
00866 * @param[in] buffer_len Size of output buffer in bytes
00867 *
00868 * @return Pointer to buffer (or "" if buffer is nullptr)
00869 *
00870 * @note Flags formatted as pipe-separated: "ACK|RETX|PRI"
00871 *
00872 * @see rx_frame_ascii.h for full documentation
00873 */
00874 const char* rx_frame_ascii_flags_str(uint8_t flags, char* buffer, uint32_t buffer_len)
00875 {
00876     if (buffer == nullptr || buffer_len == k_empty_buffer) {
00877         return "";
00878     }
00879
00880     uint32_t pos = 0;
00881
00882     if (flags == k_frame_flag_none) {
00883         pos = internal_append_str(buffer, pos, buffer_len, "NONE");
00884         buffer[pos] = '\0';
00885         return buffer;
00886     }
00887
00888     bool first = true;
00889
00890     /* Use helper to append each flag */
00891     pos =
00892         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_requires_ack, "ACK");
00893     pos =
00894         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_retransmit, "RETX");
00895     pos = internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_priority, "PRI");
00896     pos =
00897         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_fec_enabled, "FEC");
00898     pos =
00899         internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_soft_nack, "SOFT");
00900
00901     buffer[pos] = '\0';
00902     return buffer;
00903 }
00904
00905 /**
00906 * @brief Format frame as human-readable ASCII string
00907 *
00908 * @details

```



```

00909 * Implementation of rx_frame_ascii_format(). Orchestrates header, payload,
00910 * and CRC formatting using internal helper functions.
00911 *
00912 * @par Implementation Notes
00913 * - Validates all inputs before any output
00914 * - Calls internal_format_header() for metadata line
00915 * - Calls internal_format_payload() for hex dump
00916 * - Appends CRC footer directly
00917 * - Detects truncation and returns k_rx_err_no_mem
00918 *
00919 * @param[in] frame Frame to format (must not be NULL)
00920 * @param[in] is_tx Direction (true=TX, false=RX)
00921 * @param[out] output Output buffer for formatted string
00922 * @param[in] output_max_len Maximum output buffer size
00923 * @param[out] output_len Pointer to receive actual output length
00924 *
00925 * @return k_rx_ok on success
00926 * @return k_rx_err_invalid_arg if frame/output/output_len is nullptr
00927 * @return k_rx_err_no_mem if output buffer is too small
00928 *
00929 * @see rx_frame_ascii.h for full documentation
00930 */
00931 rx_err_t rx_frame_ascii_format(const rx_frame_t* frame,
00932                               bool is_tx,
00933                               char* output,
00934                               uint32_t output_max_len,
00935                               uint32_t* output_len)
00936 {
00937     /* Rule 5: Pre-condition validation */
00938     if (frame == nullptr) {
00939         return k_rx_err_invalid_arg;
00940     }
00941     if (output == nullptr) {
00942         return k_rx_err_invalid_arg;
00943     }
00944     if (output_len == nullptr) {
00945         return k_rx_err_invalid_arg;
00946     }
00947     if (output_max_len < k_min_output_size) {
00948         return k_rx_err_no_mem;
00949     }
00950
00951     /* Validate payload length does not exceed frame capacity */
00952     if (frame->header.length > k_frame_max_payload) {
00953         return k_rx_err_invalid_arg;
00954     }
00955
00956     uint32_t pos = 0;
00957
00958     /* Format header metadata */
00959     pos = internal_format_header(output, pos, output_max_len, &frame->header, is_tx);
00960
00961     /* Payload hex dump - use validated length to prevent out-of-bounds reads */
00962     pos = internal_format_payload(output, pos, output_max_len, frame->payload, frame->header.length);
00963
00964     /* CRC */
00965     pos = internal_append_str(output, pos, output_max_len, "[CRC] ");
00966     pos = internal_append_hex_u32(output, pos, output_max_len, frame->crc);
00967     pos = internal_append_str(output, pos, output_max_len, "\r\n");
00968
00969     /* Rule 5: Post-condition validation - check for truncation */
00970     /* If pos >= output_max_len, we don't have room for null terminator */
00971     if (pos >= output_max_len) {
00972         /* Buffer was too small - data was truncated */
00973         if (output_max_len > 0) {
00974             output[output_max_len - 1] = '\0'; /* Null terminate at buffer end */
00975         }
00976         return k_rx_err_no_mem;
00977     }
00978
00979     /* Null terminate */
00980     output[pos] = '\0';
00981
00982     *output_len = pos;
00983
00984     return k_rx_ok;
00985 }

```

